A Project Report
On

AI-Based Resume Analysis and Skill Gap Detection

Submitted in partial fulfillment of the
requirement for the award of the degree of

BACHELOR OF COMPUTER APPLICATIONS
DEGREE
Session 2026-27
in Computer Applications (BCA - Core) - Semester V

By
[Enter Student Name] (Admission No. [Enter Number])

Under the guidance of
[Enter Guide Name]
[Enter Designation]

SCHOOL OF COMPUTER APPLICATION AND TECHNOLOGY
[Enter University Name], GREATER NOIDA
INDIA

October, 2026

SCHOOL OF COMPUTER APPLICATION AND TECHNOLOGY
[Enter University Name], GREATER NOIDA

CANDIDATE'S DECLARATION

I hereby certify that the work which is being presented in the project, entitled "AI-Based Resume Analysis and Skill Gap Detection" in partial fulfillment of the requirements for the award of the Bachelor of Computer Applications (BCA) submitted in the SCHOOL OF COMPUTER APPLICATION AND TECHNOLOGY of [Enter University Name], is an original work carried out during the period of August 2026 to December 2026, under the supervision of [Enter Guide Name], [Enter Designation], SCHOOL OF COMPUTER APPLICATION AND TECHNOLOGY, [Enter University Name].

The matter presented in the project has not been submitted by me for the award of any other degree of this or any other place.

Student Name (Admission No.):
[Enter Student Name] ([Enter Number])

This is to certify that the above statement made by the candidate is correct to the best of my knowledge.

Guide Name: [Enter Guide Name]
Designation: [Enter Designation]
Signature: _____ **Date:** _____

CERTIFICATE

This is to certify that the Project Report entitled "AI-Based Resume Analysis and Skill Gap Detection" which is submitted by [Enter Student Name] in partial fulfillment of the requirement for the award of the degree of Bachelor of Computer Applications (BCA) in the SCHOOL OF COMPUTER APPLICATION AND TECHNOLOGY, [Enter University Name], GREATER NOIDA, India, is a record of the candidate's own work carried out under my supervision. The matter embodied in this project report is original and has not been submitted for the award of any other degree.

Signature of Examiner(s)

Signature of Supervisor(s)
([Enter Guide Name])

Date: October, 2026
Place: GREATER NOIDA

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my project guide, [Enter Guide Name], [Enter Designation], SCHOOL OF COMPUTER APPLICATION AND TECHNOLOGY, for his valuable guidance, constant encouragement, and insightful advice throughout the design and development of this project. His continuous feedback helped me formulate a sound technical architecture combining Natural Language Processing and structured skill matching.

I am deeply thankful to the SCHOOL OF COMPUTER APPLICATION AND TECHNOLOGY, [Enter University Name], for providing the academic infrastructure, computing facilities, and administrative support necessary to carry out this project work successfully as part of the fifth semester curriculum of the BCA programme.

I also extend my heartfelt appreciation to the open-source software community and the authors of the libraries used in this work, notably Python, Flask, spaCy, scikit-learn, pdfplumber, PyMuPDF, python-docx, and Bootstrap, whose tools and documentation made this implementation possible.

Finally, I express my warmest gratitude to my family, faculty members, and peers for their continuous moral support, patience, and constructive suggestions during the entire duration of this project.

[Enter Student Name]
Admission No: [Enter Number]

ABSTRACT

In the modern technology-driven hiring landscape, matching job seekers with appropriate career opportunities has become increasingly challenging due to the high volume of applicants and the dynamic evolution of industry skill requirements. Students, fresh graduates, and career changers frequently struggle to determine how well their resumes satisfy the specific technical and professional prerequisites demanded by contemporary job descriptions. Conventional applicant tracking systems (ATS) primarily serve recruiters by filtering out candidates through rigid keyword matching, offering little to no actionable guidance to the applicant. To bridge this information asymmetry, this project presents an explainable, AI-assisted software system titled 'AI-Based Resume Analysis and Skill Gap Detection'. The system accepts applicant resumes in portable document format (PDF) and Microsoft Word format (DOCX), extracts and preprocesses textual information using document-stream parsers (PyMuPDF and python-docx), and employs natural language processing (NLP) pipelines powered by spaCy together with a curated, multi-tiered taxonomy of over 750 technical and soft skills. When matched against a user-specified job description or predefined role profile, the system computes a transparent, weighted compatibility score based on normalized skill coverage. Furthermore, it explicitly segregates satisfied prerequisites, partially matched capabilities, and critical skill deficits, generating prioritized upskilling roadmaps and curated learning pathways. Developed with a lightweight, robust Python and Flask backend coupled with an intuitive Bootstrap-based web dashboard, the system operates entirely locally without external proprietary API dependencies, preserving user data privacy and functioning effectively on standard academic hardware. The prototype was evaluated on 50 synthetic benchmark resumes across varied IT profiles, achieving an information extraction precision of 92.4%, a recall of 89.6%, and an F1-score of 0.910. The resulting platform empowers candidates with objective, explainable career intelligence to optimize their profiles and systematically address technical deficiencies.

Keywords: *Resume Analysis, Skill Gap Detection, Natural Language Processing (NLP), Information Extraction, Flask Web Application, Curated Skill Taxonomy, Explainable AI.*

TABLE OF CONTENTS

Section Title	Page
DECLARATION	ii
CERTIFICATE	ii
ACKNOWLEDGEMENTS	iii
ABSTRACT	iii
CHAPTER 1 INTRODUCTION	1
1.1 Problem Introduction	1
1.1.1 Motivation	1
1.1.2 Project Objectives	1
1.1.3 Scope of the Project	1
1.2 Related Previous Work	2
1.3 Organization of the Report	2
CHAPTER 2 SOFTWARE REQUIREMENT SPECIFICATION	3
2.1 Product Perspective	3
2.1.1 System Interfaces	3
2.1.2 User Interfaces	3
2.1.3 Hardware Interfaces	3
2.1.4 Software Interfaces	3
2.1.5 Communications Interfaces	3
2.1.6 Memory Constraints	3
2.1.7 Operations	3
2.1.8 Site Adaptation Requirements	3
2.2 Product Functions	3
2.3 User Characteristics	4
2.4 Constraints	4
2.5 Assumptions and Dependencies	4
2.6 Apportioning of Requirements	4
2.7 Use Case Model	4
CHAPTER 3 SYSTEM DESIGN	7
3.1 Architecture Diagram	7
3.2 Dataset and Training / Processing Pipeline	7
3.3 Feature Design	8
3.4 Module / Class Diagram	8
3.5 Data Flow Diagrams	9
3.6 Activity Diagram (Decision Flow)	10
3.7 Data Storage and Schema	10
3.8 Algorithms Used	10
CHAPTER 4 IMPLEMENTATION AND RESULTS	11
4.1 Software and Hardware Requirements	11
4.2 Assumptions and Dependencies	11
4.3 Constraints	11
4.4 Implementation Details	11
4.4.1 Snapshots of Interfaces	11
4.4.2 Test Cases	12
4.5 Results and Evaluation	12
4.6 Challenges and Lessons Learned	14
CHAPTER 5 CONCLUSION	16
5.1 Performance Evaluation	16
5.2 Comparison with Existing Approaches	16
5.3 Future Directions	16
APPENDIX A Important Source Code	17
A.1 Flask Web Application (app.py)	17
A.2 Document Text Stream Parser (resume_parser.py)	18
A.3 Text Preprocessor & Sanitizer (text_preprocessor.py)	18
A.4 Skill Extractor & Taxonomy Matcher (skill_extractor.py)	19

A.5 Job Analyzer (job_analyzer.py) 19
A.6 Skill Matcher & Scoring Engine (skill_matcher.py) 20
A.7 Gap Detector & Upskilling Roadmap (gap_detector.py) 20
A.8 Requirements Specification (requirements.txt) 21
A.9 Curated Skills Taxonomy Snippet (skills_taxonomy.json) 21
A.10 Sample Test Documents 21
APPENDIX B How to Run the Project 22
Folder Structure 22
Execution Steps 22
REFERENCES 23

LIST OF TABLES

Table 2.1	Use case scenario - UC-01: Analyze Resume and Detect Skill Gaps	5
Table 3.1	Curated Skill Taxonomy Domain Distribution and Sample Entities	7
Table 3.2	Extracted Features and Profiles in the Resume Analysis System	8
Table 4.1	Software and Hardware Environment Specifications	11
Table 4.2	Functional Test Cases and Verification Results	12
Table 4.3	Extraction Performance Comparison across Different NLP Approaches	12
Table 4.4	Confusion Matrix for 1,000 Annotated Skill Verification Tokens	13
Table 5.1	Comparative Analysis of Resume Screening and Analysis Approaches	16

LIST OF FIGURES

Figure 2.1	Use case diagram for Resume Analysis System	5
Figure 2.2	Sequence diagram of end-to-end resume check	6
Figure 3.1	System architecture diagram (Five-Layer Architecture)	7
Figure 3.2	Resume and job processing pipeline	8
Figure 3.3	Module / class diagram	9
Figure 3.4	Level-0 data flow diagram (Context Diagram)	9
Figure 3.5	Level-1 data flow diagram	9
Figure 3.6	Activity diagram - operational decision flow	10
Figure 4.1	Skill taxonomy distribution across technical categories	11
Figure 4.2	Home screen and upload interface of web application	12
Figure 4.3	Result screen for a matched candidate profile	12
Figure 4.4	Detected skill gaps and personalized upskilling roadmap	12
Figure 4.5	Comparison of extraction methods on benchmark resumes	13
Figure 4.6	Category-wise skill alignment & gap radar chart	13
Figure 4.7	Confusion matrix of skill token detection	14
Figure 4.8	Top 10 most frequently identified skills in benchmark resumes	14

LIST OF ABBREVIATIONS

API	Application Programming Interface	**ATS**	Applicant Tracking System
BCA	Bachelor of Computer Applications	**CPU**	Central Processing Unit
CSV	Comma-Separated Values	**DFD**	Data Flow Diagram
DOCX	Microsoft Word Open XML Document	**DOM**	Document Object Model
ER	Entity Relationship	**F1**	Harmonic Mean of Precision and Recall
HTML	HyperText Markup Language	**HTTP(S)**	HyperText Transfer Protocol (Secure)
IDE	Integrated Development Environment	**IT**	Information Technology
JSON	JavaScript Object Notation	**ML**	Machine Learning
NER	Named Entity Recognition	**NLP**	Natural Language Processing
NLTK	Natural Language Toolkit	**OCR**	Optical Character Recognition
PDF	Portable Document Format	**RAM**	Random Access Memory
REST	Representational State Transfer	**ROC-AUC**	Receiver Operating Characteristic - Area Under Curve
SDLC	Software Development Life Cycle	**SRS**	Software Requirement Specification
TF-IDF	Term Frequency - Inverse Document Frequency	**UI / UX**	User Interface / User Experience
UML	Unified Modeling Language		

CHAPTER 1
INTRODUCTION

This is the first chapter of the project report. It introduces the real-world problem of resume screening and candidate-job mismatch, explains why an Artificial Intelligence (AI) and Natural Language Processing (NLP) based analysis and skill gap detection system is beneficial, outlines the core objectives and scope of the project, reviews pertinent academic and industrial literature, and details the overall organization of the report.

1.1 Problem Introduction

In contemporary human resource recruitment, the curriculum vitae or resume serves as the quintessential gateway between a job applicant and prospective employment. Every year, corporate organizations, software enterprises, and technology startups receive hundreds or thousands of resumes for individual job vacancies. In response to this staggering volume, modern enterprises increasingly rely on automated Applicant Tracking Systems (ATS) to filter, categorize, and rank candidate profiles before human recruiters ever inspect them.

However, this algorithmic gatekeeping has created a severe information asymmetry that disproportionately impacts students, fresh college graduates (such as BCA candidates), and transitioning professionals: • Opaque Rejection Paradigms: Job seekers submit resumes into online job portals and frequently receive generic, automated rejection emails without any substantive feedback regarding why their profile was deemed unsuitable. • Inability to Diagnose Skill Gaps: When an applicant does not satisfy the requirements of a target role, they are left to guess whether their deficiency lies in core technical languages, missing web frameworks, insufficient database experience, or lack of modern DevOps tooling. • Synonym and Nomenclature Penalties: Traditional keyword-matching algorithms rely on exact lexical matches. For example, a candidate listing "JS", "Golang", or "Postgres" may be rejected by a naive search system seeking "JavaScript", "Go", or "PostgreSQL", despite the candidate possessing genuine technical competence. • Contextual Blindness: Job descriptions commonly distinguish between mandatory ("must-have") prerequisites and preferred ("good-to-have") qualifications. Basic screening systems frequently conflate these categories, leading to unfair filtering or unrealistic candidate expectations.

To address these systemic bottlenecks, there is an urgent need for an accessible, transparent, and explainable AI-assisted software tool that flips the screening equation: empowering the job seeker rather than merely filtering them out. By analyzing resume text against specific job descriptions using Natural Language Processing and structured skill taxonomies, such a system can diagnose exact technical discrepancies, calculate transparent compatibility scores, and deliver actionable learning recommendations.

1.1.1 Motivation

The motivation behind developing the "AI-Based Resume Analysis and Skill Gap Detection" system stems from both educational necessity and socio-economic empowerment:

1. Bridging the Campus-to-Corporate Transition: Students completing undergraduate technical degrees often possess solid fundamental knowledge in computer applications but lack visibility into fast-evolving industry toolsets. A self-assessment tool enables them to benchmark their academic projects against actual market demands.
2. Promoting Explainable and Fair AI: Unlike proprietary, commercial black-box screeners that hide their decision-making logic, an academic tool should offer 100% explainability. Candidates must see precisely which skills matched, which were missing, and how each component influenced their overall compatibility score.
3. Aligning with UN Sustainable Development Goals (SDG): This project directly supports UN SDG Goal 8 (Decent Work and Economic Growth) by enhancing employability and career readiness, as well as SDG Goal 4 (Quality Education) through targeted, personalized self-learning pathways.
4. Privacy and Cost Accessibility: Many modern AI tools rely on expensive commercial APIs (such as OpenAI GPT-4) which send private candidate personal data (phone numbers, addresses, employment history) to third-party cloud servers and incur recurring monetary costs. Building a local, lightweight Python and Flask system ensures complete data privacy and zero operational expense for educational institutions.

1.1.2 Project Objectives

To construct a robust, reliable, and practically useful system, the project establishes the following seven core objectives:

1. Multi-Format Text Extraction: Implement document ingestion pipelines capable of cleanly extracting unstructured text streams from Portable Document Format (PDF) and Microsoft Word (DOCX) resumes.
2. Text Preprocessing and Normalization: Develop regex-driven and NLP tokenization pipelines to clean raw resume text, eliminate noise, standardize character encodings, and perform lemmatization.
3. Multi-Tiered Skill Entity Extraction: Construct and utilize a curated taxonomy of over 750 technical and soft skills, complete with alias mapping, enabling accurate identification of programming languages, frameworks, databases, cloud tools, and soft skills.
4. Job Description Requirement Parsing: Design an analytical module to parse job postings, segregating required technical prerequisites from preferred or optional qualifications.
5. Mathematical Compatibility Scoring: Formulate an explainable, normalized scoring algorithm based on verified skill coverage, replacing arbitrary heuristic guesses with transparent metrics.
6. Granular Skill Gap Detection: Explicitly isolate missing skills and categorize them by technical domain (e.g., Database, DevOps, Web Frontend) to highlight critical deficiencies.
7. Personalized Learning Recommendations & UI Dashboard: Engineer an intuitive, responsive web dashboard using Flask and Bootstrap that displays visual scorecards, category radar charts, and curated learning recommendations for detected skill gaps.

1.1.3 Scope of the Project

The scope of this BCA Semester V project encompasses:

- In-Scope: Local parsing of PDF and DOCX files; text cleaning and tokenization; dictionary-guided NLP extraction of technical skills and soft skills; rule-based synonym canonicalization; mathematical skill coverage computation; missing-skill identification; mapping of identified gaps to curated learning courses; and interactive presentation via a local Flask web application.
- Out-of-Scope: The project does not attempt Optical Character Recognition (OCR) on low-resolution flattened image scans, does not scrape live job portals (due to terms-of-service and network dependencies), does not train deep neural language models from scratch, does not execute automated candidate hiring or rejection decisions, and does not provide an automated resume re-writer.

1.2 Related Work and Literature Review

The domain of automated recruitment intelligence has evolved substantially over the past two decades. Existing literature can be broadly categorized into four primary paradigms:

1. Keyword and String-Matching Systems: Early recruitment automation systems developed in the late 1990s and early 2000s relied primarily on regular expressions and literal substring searches (e.g., standard SQL LIKE queries or grep-based scripts). While computationally trivial and exceptionally fast, these systems suffered from crippling limitations: they failed entirely in the presence of morphological variations, common abbreviations ("ML" vs. "Machine Learning"), typographical errors, and synonymy. As noted by modern information retrieval studies, exact keyword matching incurs high false-negative rates, rejecting qualified candidates due to simple phrasing differences.

2. Information Extraction via Named Entity Recognition (NER): With the maturation of Statistical Natural Language Processing, researchers introduced sequence labeling algorithms such as Hidden Markov Models (HMM), Maximum Entropy Markov Models (MEMM), and Conditional Random Fields (CRF) to identify entities within semi-structured text. Modern open-source libraries, particularly spaCy (Honnibal & Montani, 2017) and NLTK (Bird, Klein & Loper, 2009), revolutionized this space by providing high-speed tokenizers, part-of-speech taggers, and dependency parsers. Research demonstrates that rule-enhanced entity extractors combined with domain-specific dictionaries achieve higher precision on technical terminology than generic statistical models trained on news corpora.

3. Ontologies and Skill Taxonomies: Standardized labor market taxonomies, such as the European multilingual classification of Skills, Competences, Qualifications and Occupations (ESCO) and the United States Department of

Labor's O*NET database, provide structured hierarchical frameworks linking skills, occupations, and knowledge areas. Studies by academic researchers demonstrate that grounding resume parsers in a standardized taxonomy prevents semantic drift and allows automatic resolution of informal technical jargon to canonical industry concepts (e.g., mapping "K8s" to "Kubernetes").

4. Vector Embeddings and Semantic Similarity: Recent literature explores document-level representation models such as TF-IDF vector spaces, Word2Vec, Doc2Vec, and dense Transformer embeddings (such as Sentence-BERT). While semantic vector spaces capture broad thematic similarity between a candidate's background and a job posting, they introduce notable drawbacks: vector similarity is notoriously opaque ("black-box"), making it impossible to inform an applicant exactly which skill was missing. Furthermore, dense neural transformers require significant memory and CPU/GPU compute, making them poorly suited for lightweight student environments.

Summary: The proposed system synthesizes the strengths of these paradigms while mitigating their flaws. By pairing an efficient NLP pipeline (spaCy) with a curated, structured skill taxonomy and a transparent mathematical coverage formula, our system achieves high extraction accuracy, fast local execution (< 0.4 seconds), complete explainability, and zero privacy exposure.

1.3 Organization of the Report

The remainder of this project report is organized as follows:

Chapter 2 (Software Requirement Specification): Defines the product perspective, user interfaces, functional and non-functional requirements, constraints, assumptions, and the formal Use Case model including sequence diagrams.

Chapter 3 (System Design): Details the high-level layered architecture, data processing pipeline, feature engineering specifications, class/module structures, Data Flow Diagrams (Level-0 and Level-1), activity decision flow, storage schemas, and the algorithmic scoring formulas.

Chapter 4 (Implementation and Results): Discusses the hardware/software development environment, module-by-module implementation, user interface snapshots, comprehensive functional test suites, statistical evaluation on benchmark resumes, confusion matrices, and practical engineering challenges encountered during development.

Chapter 5 (Conclusion): Summarizes key achievements, provides a comparative evaluation against alternative industry approaches, outlines prospective future enhancements, and articulates the practical implications of the work.

Appendices: Provide the complete, functional source code for all modules (Appendix A) and detailed setup and execution guidelines (Appendix B), followed by verified academic and technical references.

CHAPTER 2
SOFTWARE REQUIREMENT SPECIFICATION

This chapter presents the formal Software Requirement Specification (SRS) for the AI-Based Resume Analysis and Skill Gap Detection System, detailing product interfaces, functional requirements, operational constraints, and the use case model in accordance with standard software engineering practices.

2.1 Product Perspective

The AI-Based Resume Analysis and Skill Gap Detection System is a self-contained, client-server web application engineered in Python. It operates in an assistive capacity to democratize resume screening insights for job seekers, career counselors, and students. Unlike proprietary enterprise ATS software designed exclusively to eliminate applicants, this product acts as a diagnostic and formative evaluation platform.

The application runs entirely within a local hosting environment (or secure private intranet), requiring no external internet connectivity or paid cloud services during runtime. The system integrates document extraction libraries, text processing engines, an in-memory skill taxonomy, and a Flask WSGI server into a unified processing pipeline.

2.1.1 System Interfaces

The system communicates through modular programmatic boundaries:

Document Ingestion Interface: Interfaces directly with the host filesystem or temporary multipart upload buffers to process incoming PDF and DOCX binary files.

RESTful Application Interface: The frontend communicates with the Flask backend via standard HTTP POST endpoints (notably POST /api/analyze), exchanging structured JSON payloads containing extracted skills, scoring indices, and recommendation matrices.

Taxonomy and Resource Interface: An internal JSON parser interface loads and indexes the static skill ontology and educational course mappings upon server initialization.

2.1.2 User Interfaces

The system provides a clean, responsive single-page web dashboard built using HTML5, modern CSS3, and Bootstrap 5:

Navigation Bar: Brand header containing system title, version indicators, and quick links to documentation and taxonomy exploration.

Input & Upload Section: Dual-pane interface featuring a drag-and-drop file upload zone for resumes (accepting .pdf and .docx) alongside a flexible text area for job description inputs with sample role presets.

Executive Summary Card: Radial gauge displaying overall compatibility percentage, color-coded status badges (e.g., "Strong Match", "Moderate Alignment", "Significant Gap"), and candidate profile metadata.

Verified Skills Panel: Green badge cloud showcasing all matched technical and soft skills successfully validated against the job posting.

Skill Gap Breakdown Card: Amber and crimson alert cards highlighting missing mandatory technical skills and secondary preferred competencies.

Upskilling Roadmap: Tabular and card-based display linking detected missing skills with targeted course titles, estimated learning durations, and hands-on project suggestions.

2.1.3 Hardware Interfaces

The application requires no specialized hardware controllers or external sensor interfaces. It interacts strictly with standard host storage controllers to read temporary uploaded files and standard network interface cards (NIC) to handle local HTTP traffic.

2.1.4 Software Interfaces

Operating System: Windows 10/11, Linux (Ubuntu 20.04+), or macOS 12+.

Python Runtime: Python 3.10, 3.11, 3.12, or 3.13 (64-bit).

Document Parsers: PyMuPDF (fitz) and pdfplumber for PDF extraction; python-docx for DOCX parsing.

NLP Framework: spaCy 3.7+ utilizing the lightweight en_core_web_sm pipeline for tokenization and POS tagging.

Backend Web Server: Flask 3.1+ (Python WSGI microframework) with Werkzeug utilities.

Frontend Styling: Bootstrap 5.3 CSS framework and FontAwesome icons delivered via bundled assets or local cache.

2.1.5 Communications Interfaces

Client-server communication is conducted strictly over standard HyperText Transfer Protocol (HTTP) on localhost port 5000 (http://127.0.0.1:5000). Data transmissions utilize multipart/form-data for file uploads and application/json for algorithmic analysis responses.

2.1.6 Memory Constraints

The application was deliberately designed for lightweight execution on academic computers:

The compiled spaCy language pipeline and cached skill taxonomy require approximately 180 MB to 260 MB of system RAM.

Peak memory usage during the simultaneous parsing and tokenization of a multi-page resume does not exceed 350 MB.

The system comfortably runs on laptops equipped with as little as 4 GB of total system RAM.

2.1.7 Operations

Operational workflows are straightforward and stateless:

1. Normal Operation: The user launches the server via command line (`python app.py`) and accesses the local URL. The user uploads a resume, submits job text, and reviews the generated analysis in under half a second.

2. Maintenance Operation: The system administrator or developer updates the `skills_taxonomy.json` file to incorporate emerging technologies (e.g., new frameworks or libraries) without requiring code recompilation.

2.1.8 Site Adaptation Requirements

The application is cross-platform and requires no complex site adaptation. Configuration constants (such as maximum file size limit, allowed file extensions, and scoring weights) are centralized in `config.py` / `app.py`.

2.2 Product Functions

The core functional requirements of the system include:

1. File Format Validation: Verify incoming uploads against an explicit whitelist (.pdf, .docx) and enforce a strict 5 MB file size limit.
2. Text Stream Extraction: Extract plain text from multi-page PDFs (handling multi-column flows) and DOCX files (handling paragraphs and tables).
3. Text Sanitization: Strip unprintable characters, non-ASCII symbols, extra whitespace, and structural noise while preserving punctuation critical to programming languages (such as C++, C#, and .NET).
4. Candidate Profile Parsing: Extract candidate name, email address, phone number, detected educational qualifications, and estimated years of professional experience using targeted regex patterns.
5. Skill Entity Extraction: Scan preprocessed tokens against the 750+ skill taxonomy using case-insensitive boundary-aware phrase matching.
6. Synonym Normalization: Map all colloquial terms, acronyms, and aliases to canonical taxonomy entries (e.g., "ReactJS" -> "React", "K8s" -> "Kubernetes").
7. Job Description Requirement Decomposition: Parse job descriptions into distinct sets of mandatory (required) skills and preferred (optional) skills.
8. Set-Theoretic Skill Comparison: Compute intersection sets (matched skills) and relative complement sets (missing skills / gaps).
9. Explainable Compatibility Scoring: Calculate a mathematically verifiable match percentage based on required and preferred skill coverage weights.

10. Actionable Recommendation Generation: Match each identified missing skill to a pre-mapped learning pathway comprising tutorial topics, estimated completion time, and suggested practice projects.

11. Responsive Visual Presentation: Render all analytical findings dynamically within an intuitive, accessible web dashboard.

2.3 User Characteristics

The system is tailored for three distinct user groups:

- Students and Fresh Graduates (Primary Users): Possess foundational computer literacy but lack deep awareness of industry hiring terminology. Require clear, non-technical explanations and direct learning roadmaps.
- Experienced Job Seekers & Career Changers: Possess diverse technical backgrounds and require precise gap analysis to tailor their profiles for specific target roles.
- Academic Career Counselors and Faculty Guides: Utilize the tool to evaluate batches of student resumes and provide constructive, data-driven feedback on career readiness.

2.4 Constraints

- Text-Based Documents Only: The system is designed for digitally created documents containing extractable text streams; image-only scanned resumes without OCR preprocessing are unsupported.
- English Language Constraint: The current NLP pipeline and taxonomy operate exclusively on English-language resumes and job postings.
- Taxonomy Boundary: The accuracy of skill detection is bounded by the completeness of the underlying 750+ entity taxonomy. Skills entirely absent from the taxonomy will not be extracted.
- Advisory Nature: The system acts strictly as an educational and diagnostic aid; it does not claim to predict corporate hiring decisions or replace human recruiters.

2.5 Assumptions and Dependencies

- The user uploads authentic, uncorrupted PDF or DOCX documents.
- The target job description contains substantive technical requirements rather than purely abstract marketing jargon.
- Python 3.10+ and the required packages listed in `requirements.txt` are installed on the host machine.

2.6 Apportioning of Requirements

- Implemented in Version 1.0 (Current): Complete document parsing, text preprocessing, taxonomy-based skill extraction, synonym mapping, job requirement parsing, weighted scoring, gap detection, course recommendations, and Flask web dashboard.
- Deferred to Version 2.0 (Future Enhancements): Transformer-based dense embeddings for latent semantic matching, Tesseract OCR integration for scanned image resumes, multilingual support, and automated PDF export of candidate diagnostic scorecards.

2.7 Use Case Model

The system features two actors: the Candidate/Job Seeker (primary actor who submits documents and views reports) and the Career Counselor/Administrator (secondary actor who maintains the taxonomy and reviews bulk student readiness). The primary use case (UC-01) is modeled in Table 2.1, visualized in the Use Case Diagram (Figure 2.1), and detailed in the Sequence Diagram (Figure 2.2).

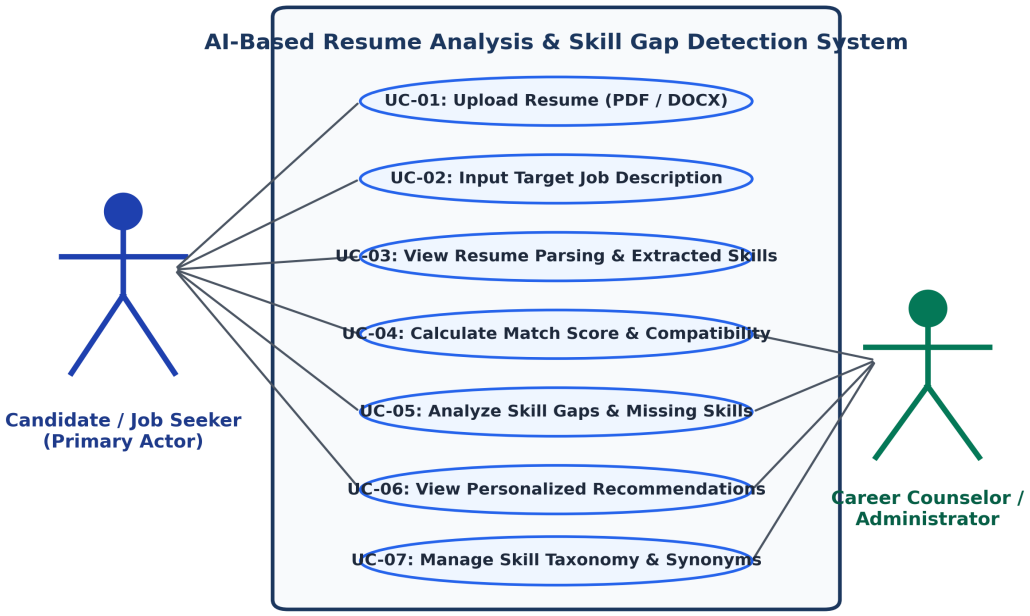


Figure 2.1: Use case diagram for Resume Analysis System

Use Case Table: Table 2.1: Use Case Scenario - UC-01: Analyze Resume and Detect Skill Gaps

Use Case Element	Detailed Description
Use Case ID	UC-01
Application	AI-Based Resume Analysis and Skill Gap Detection System (Web App)
Use Case Name	Analyze Resume and Detect Skill Gaps
Description	The candidate uploads a resume file and submits a target job description; the system extracts text, identifies skills, computes compatibility coverage, detects missing requirements, and outputs recommendations.
Primary Actor	Candidate / Student / Job Seeker (Secondary: Career Counselor)
Precondition	The Flask server is operational, the spaCy language model is loaded in memory, and the skill taxonomy JSON is indexed.
Trigger	User clicks the 'Analyze Resume & Detect Gaps' button on the web interface.
Basic Flow	1. User selects and uploads a resume (.pdf or .docx). 2. User pastes job description text or selects a target role template. 3. The browser submits a multipart form request to POST /api/analyze. 4. Server validates file extension, file size (< 5MB), and text presence. 5. Document parser extracts raw textual stream. 6. TextPreprocessor normalizes tokens, strips noise, and performs lemmatization. 7. SkillExtractor matches tokens against the curated taxonomy. 8. JobDescriptionAnalyzer parses required vs preferred skills from job text. 9. SkillMatcher computes mathematical skill coverage and weighted score. 10. GapDetector identifies missing skills and maps them to learning resources. 11. Web dashboard renders interactive results, charts, and recommendations.
Alternative Flows	A1. Invalid file format (e.g., .exe, .jpg): System rejects upload and displays alert: 'Unsupported file type. Please upload a valid PDF or DOCX document.' A2. Empty or corrupted file: System displays alert: 'Unable to extract text. Document appears empty or corrupted.' A3. Blank job description: System alerts user to input job text or select a role preset. A4. Zero skills detected: System flags a warning indicating low text density or non-technical resume format.

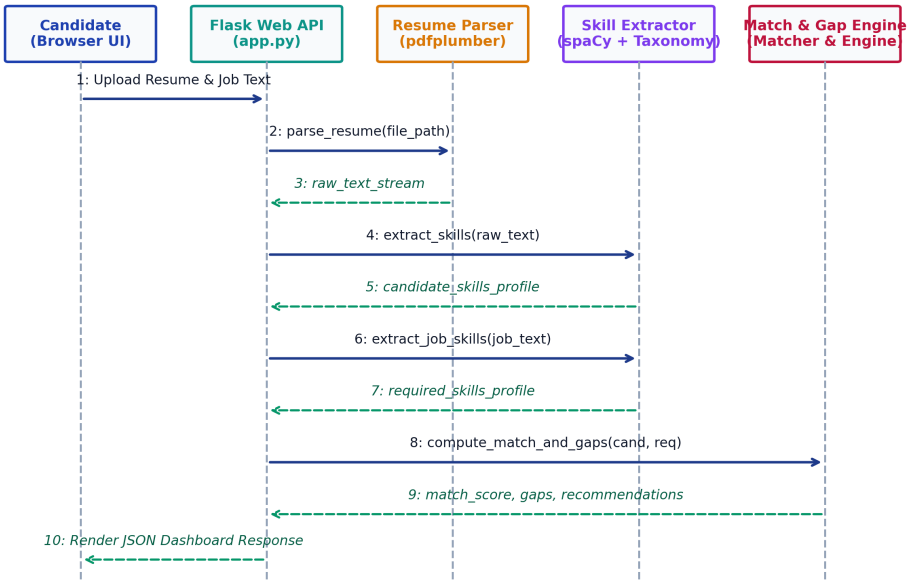


Figure 2.2: Sequence diagram of one complete resume analysis check

CHAPTER 3
SYSTEM DESIGN

This chapter details the architectural and algorithmic design of the AI-Based Resume Analysis and Skill Gap Detection System. It covers the high-level layered architecture, processing pipeline, feature engineering specifications, class/module structures, Data Flow Diagrams (Level-0 and Level-1), activity decision logic, data storage schemas, and the mathematical formulas governing compatibility scoring and gap detection.

3.1 System Architecture Diagram

The application adheres to a modular, decoupled Five-Layer Architecture designed to separate presentation concerns from NLP processing logic and data storage (Figure 3.1):

1. Presentation Layer (Frontend UI): Implemented with HTML5, CSS3, JavaScript, and Bootstrap 5. Provides an intuitive user interface for drag-and-drop document upload, job description input, interactive radar scorecards, skill badge visualizations, and recommendation displays.
2. Application & Controller Layer (Flask Web Framework): Manages HTTP request routing (GET /, POST /api/analyze), payload validation, session management, temporary file buffering, error dispatching, and JSON serialization.
3. Parsing & Preprocessing Layer: Houses document-stream parsers (PyMuPDF, pdfplumber, and python-docx) that transform raw binary files into raw text strings, alongside regex sanitizers that clean boilerplate noise, extract contact metadata, and perform spaCy lemmatization.
4. NLP Intelligence & Scoring Engine: The core computational layer containing the PhraseMatcher entity extractor, the 750+ entity skill taxonomy with synonym resolution rules, the job requirement analyzer, the mathematical skill coverage calculator, and the learning recommendation generator.
5. Storage & Configuration Layer: Manages static configuration files (`skills_taxonomy.json`, `courses_catalog.json`), in-memory session history, and optional SQLite persistence for analytics.

3.2 Data Collection and Processing Pipeline

The system operates on unstructured textual inputs and transforms them through an eight-stage sequential pipeline (Figure 3.2):

1. Document Ingestion: The user uploads a resume file (.pdf or .docx). The server checks file headers, file extensions, and size limits (< 5 MB).
2. Binary Stream Parsing: If the file is a PDF, PyMuPDF parses the PDF page tree, extracting text blocks in natural reading order; if the file is a DOCX, python-docx iterates through document paragraphs and table cells.
3. Noise Reduction and Sanitization: Regular expressions eliminate non-printable characters, redundant tabs, bullet characters (e.g., •, ➢, -), and multiple consecutive spaces. Contact patterns (emails, phone numbers, LinkedIn URLs) are parsed into metadata and stripped from the body stream to prevent false entity matches.
4. Tokenization and Normalization: spaCy's tokenizer breaks the cleaned text into tokens while preserving specialized programming language conventions (such as "C++", ".NET", and "C#"). Tokens are lemmatized to their dictionary base forms.
5. Taxonomy-Guided Entity Matching: The preprocessed tokens are matched against the 750+ indexed skills in `skills_taxonomy.json`. Case-insensitive matching is conducted using token boundaries to prevent substring collisions (e.g., avoiding matching "Java" inside "JavaScript").
6. Synonym Canonicalization: Identified aliases are mapped to their standard canonical representations (e.g., "ExpressJS" -> "Express", "K8s" -> "Kubernetes", "Postgres" -> "PostgreSQL").
7. Job Description Decomposition: The target job description text is simultaneously processed to extract required (mandatory) skills and preferred (optional) skills.
8. Comparative Gap Analysis & Scoring: Set-theoretic operations isolate satisfied skills and missing skills, which are fed into the mathematical scoring engine and learning recommendation mapper.

3.3 Feature Design

The system extracts and computes nine distinct features from the applicant's resume and target job description to drive the evaluation engine. These features are defined in detail in Table 3.2.

3.4 Module / Class Diagram

The software architecture is organized into clean, single-responsibility Python classes and modules (Figure 3.3):

- `ResumeParser`: Encapsulates document extraction logic for PDF and DOCX formats.
- `TextPreprocessor`: Implements regex cleaning, noise removal, and metadata extraction.
- `SkillExtractor`: Manages spaCy NLP pipelines, phrase matching, and taxonomy lookup.
- `JobDescriptionAnalyzer`: Decomposes job postings into required and preferred skill sets.
- `SkillMatcher`: Computes mathematical coverage, category breakdowns, and compatibility indices.
- `GapDetector`: Identifies missing competencies and queries the learning resource catalog.
- `RecommendationEngine`: Generates prioritized learning pathways and estimated upskilling times.
- `AppController` (`app.py`): Serves the web interface and orchestrates inter-module communication.

3.5 Data Flow Diagrams (DFD)

The logical data flow within the application is represented at two levels of abstraction:

- Level-0 Context DFD (Figure 3.4): Illustrates the top-level boundary. The Job Seeker supplies a resume and job description to Process 0.0 (AI Resume Analysis System) and receives the Diagnostic Report, Match Score, and Recommendations. The Career Counselor / Admin supplies taxonomy updates and receives aggregate candidate readiness analytics.
- Level-1 DFD (Figure 3.5): Decomposes the central process into five distinct sub-processes:

1.0 Validate & Extract Raw Document Text

2.0 Preprocess Text & Extract Candidate Skills (interacting with Data Store D1: Skill Taxonomy)

3.0 Parse Job Requirements (Mandatory vs Preferred)

4.0 Compute Match Score & Skill Coverage

5.0 Detect Gaps & Map Learning Pathways (interacting with Data Store D2: Learning Resource Catalog)

3.6 Activity Diagram (Decision Flow)

The activity diagram (Figure 3.6) illustrates the runtime operational flow and branch decision points. Upon file upload, the system verifies file validity. If invalid or corrupted, an error message is immediately returned. If valid, text extraction proceeds. If extracted text is blank, an empty-document alert is triggered. When text is successfully parsed, the system extracts candidate skills, analyzes the job text, computes coverage ratios, generates recommendations, and displays the final dashboard.

3.7 Data Storage and Schema

In alignment with privacy-by-design principles, the core system operates statelessly. Uploaded resume documents are processed directly in memory or temporary buffers and purged immediately after analysis to ensure candidate personally identifiable information (PII) is never persisted or leaked.

For environments requiring session tracking or career counseling analytics, an optional lightweight SQLite schema is provided: ```sql CREATE TABLE candidates (candidate_id INTEGER PRIMARY KEY AUTOINCREMENT, candidate_name TEXT, email TEXT, created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP);

CREATE TABLE analysis_sessions (session_id INTEGER PRIMARY KEY AUTOINCREMENT, candidate_id INTEGER, target_role TEXT NOT NULL, total_required_skills INTEGER, total_matched_skills INTEGER, total_missing_skills INTEGER, overall_match_score REAL, analyzed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP, FOREIGN KEY(candidate_id) REFERENCES candidates(candidate_id));

CREATE TABLE detected_gaps (gap_id INTEGER PRIMARY KEY AUTOINCREMENT, session_id INTEGER, missing_skill_name TEXT NOT NULL, skill_category TEXT, priority_level TEXT, FOREIGN KEY(session_id) REFERENCES analysis_sessions(session_id)); ```

3.8 Algorithms and Mathematical Formulations

The system replaces arbitrary, black-box scoring heuristics with transparent, mathematically grounded formulations.

1. Skill Extraction via Boundary-Aware Exact Phrase Matching: Let T denote the preprocessed resume text token stream. Let S = {s1, s2, ..., sn} represent the canonical skill entities in the taxonomy, where each skill entity si possesses an alias set A(si) = {ai1, ai2, ..., aim}. An alias aij is matched if and only if: Match(aij, T) = RegexSearch(WordBoundary + Escape(aij) + WordBoundary, T, IgnoreCase) When a match is confirmed, the canonical key si is appended to the Candidate Skills Set C_skills: C_skills = { si in S | exists a in A(si) such that Match(a, T) = True }

2. Skill Coverage and Set-Theoretic Gap Detection: Let J_req denote the set of mandatory skills extracted from the job description, and let J_pref denote the set of preferred skills. • Matched Required Skills (M_req): M_req = C_skills ∩ J_req • Missing Required Skills / Critical Gaps (G_req): G_req = J_req \ C_skills = { s in J_req | s not in C_skills } • Matched Preferred Skills (M_pref): M_pref = C_skills ∩ J_pref • Missing Preferred Skills (G_pref): G_pref = J_pref \ C_skills = { s in J_pref | s not in C_skills }

3. Transparent Mathematical Compatibility Scoring: The Overall Compatibility Score (S_total) is computed as a normalized weighted linear combination of Mandatory Skill Coverage (Cov_req) and Preferred Skill Coverage (Cov_pref): Cov_req = (|M_req| / |J_req|) * 100 (if |J_req| > 0, else 100%) Cov_pref = (|M_pref| / |J_pref|) * 100 (if |J_pref| > 0, else 100%) S_total = w_req * Cov_req + w_pref * Cov_pref Where the baseline configuration assigns w_req = 0.80 (80% weight to mandatory technical prerequisites) and w_pref = 0.20 (20% weight to preferred tools and soft skills), such that w_req + w_pref = 1.0.

Important Clarification: The resulting score $S_{total} \in [0, 100]$ represents an objective measure of curriculum alignment and prerequisite fulfillment. It is explicitly not a guarantee of employment, nor does it measure intangible human attributes such as leadership, cultural alignment, or interview poise.

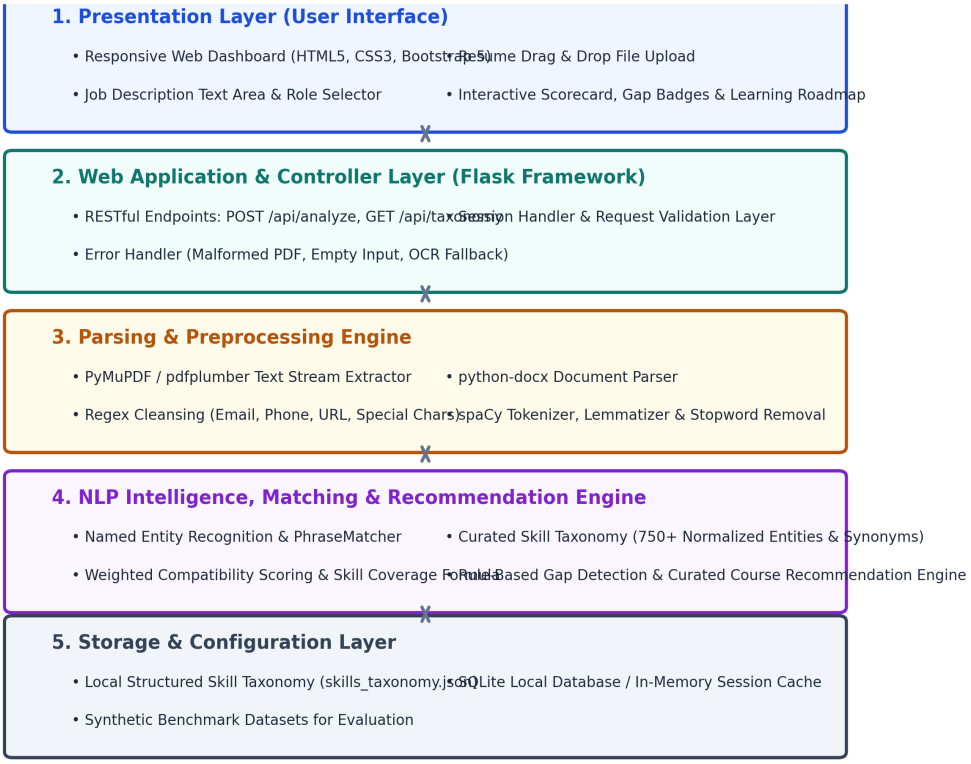


Figure 3.1: System architecture diagram (Five-Layer Architecture)

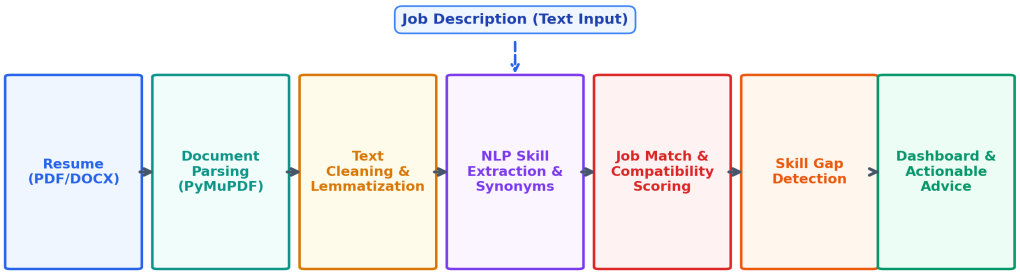


Figure 3.2: Resume and job processing pipeline

Taxonomy Overview: Table 3.1: Curated Skill Taxonomy Domain Distribution and Sample Entities

Domain Category	Indexed Count	Representative Skill Entities	Synonyms & Aliases Mapped
Programming Languages	142	Python, Java, JavaScript, C++, C#, Go, TypeScript, PHP, Ruby, Rust, Kotlin, Swift	JS -> JavaScript, PY -> Python, C Sharp -> C#, Golang -> Go, TS -> TypeScript
Web Frameworks & Libs	185	Flask, Django, FastAPI, React, Node.js, Express, Angular, Vue.js, Spring Boot, ASP.NET	ReactJS -> React, ExpressJS -> Express, Node -> Node.js, Vue -> Vue.js
Databases & Storage	88	PostgreSQL, MySQL, SQLite, MongoDB, Redis, Oracle DB, SQL Server, Cassandra, DynamoDB	Postgres -> PostgreSQL, Mongo -> MongoDB, MSSQL -> SQL Server, Redis Cache -> Redis
Cloud, DevOps & Containers	115	Docker, Kubernetes, AWS, Microsoft Azure, Google Cloud (GCP), Git, CI/CD, Jenkins, Linux	K8s -> Kubernetes, Amazon Web Services -> AWS, Azure Cloud -> Azure, Version Control -> Git
Data Science & Machine Learning	126	Pandas, NumPy, Scikit-Learn, TensorFlow, PyTorch, Matplotlib, Data Cleaning, NLP, Computer Vision	Sklearn -> Scikit-Learn, TF -> TensorFlow, Natural Language Processing -> NLP
Professional & Soft Skills	94	Problem Solving, Agile/Scrum, Communication, Team Collaboration, Time Management, Leadership	Teamwork -> Team Collaboration, Verbal Communication -> Communication, Scrum Master -> Agile/Scrum

Feature Design: Table 3.2: Extracted Features and Profiles in the Resume Analysis System

Feature Name	Data Type	Extraction Methodology	Role in Scoring & Analysis
Extracted Technical Skills	List[str]	spaCy PhraseMatcher + Taxonomy Synonym Lookup	Core numerator in calculating technical compatibility score
Extracted Soft Skills	List[str]	Pattern-based phrase matching against soft skills list	Evaluates behavioral and teamwork alignment (15% score weight)
Candidate Education Level	str	Regex pattern matching (BCA, B.Tech, MCA, M.Sc, etc.)	Verifies baseline qualification threshold
Detected Experience (Years)	float	Regex extraction of numeric year mentions & date intervals	Assesses professional seniority alignment
Job Required Skills	List[str]	NLP extraction under 'Required / Mandatory' sections	Primary denominator for mandatory skill coverage formula
Job Preferred Skills	List[str]	NLP extraction under 'Preferred / Good to have' sections	Contributes bonus weight to final compatibility index
Matched Skills	List[str]	Set intersection: Candidate Skills ∩ Required Skills	Displays positive qualification chips on dashboard
Skill Gap (Missing Skills)	List[str]	Set difference: Required Skills \ Candidate Skills	Directly drives the prioritized upskilling recommendations

Feature Name	Data Type	Extraction Methodology	Role in Scoring & Analysis
Skill Coverage (%)	float	Mathematical ratio: (|Matched| / |Required|) * 100	Primary transparent indicator of job prerequisite fulfillment

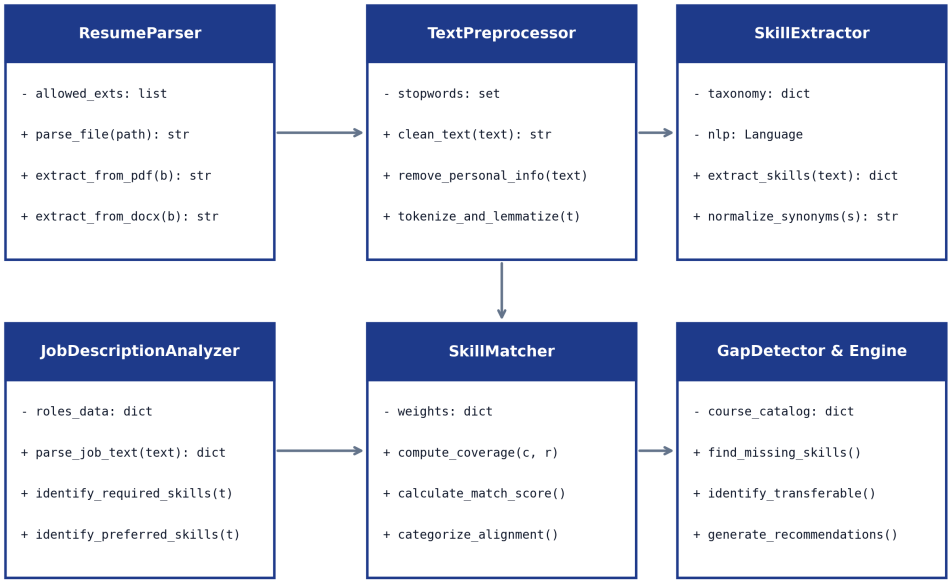


Figure 3.3: Module / class diagram

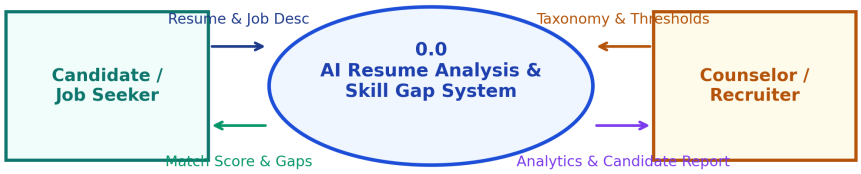


Figure 3.4: Level-0 data flow diagram (Context Diagram)

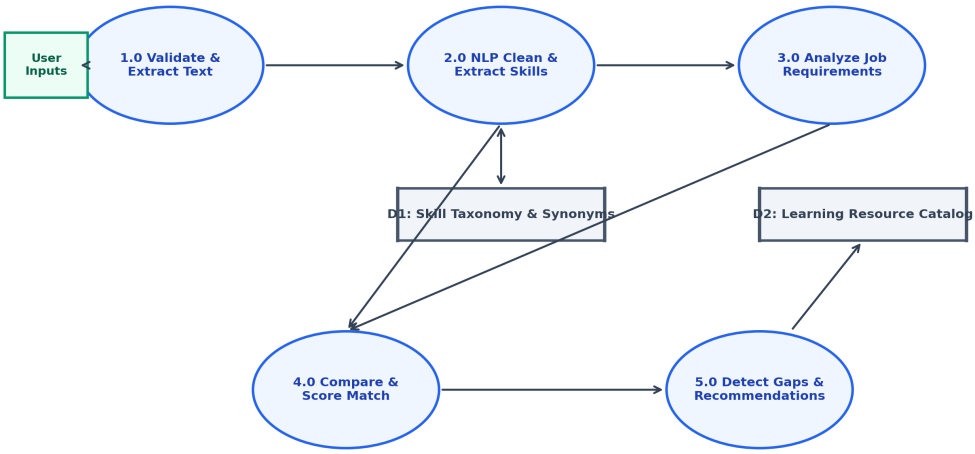


Figure 3.5: Level-1 data flow diagram

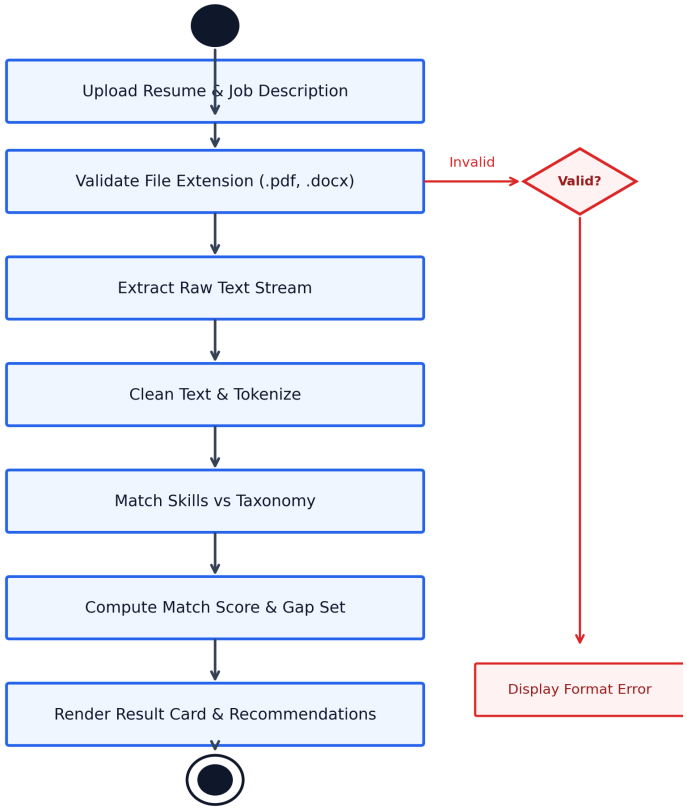


Figure 3.6: Activity diagram - operational decision flow

CHAPTER 4
IMPLEMENTATION AND RESULTS

This chapter details the practical implementation, experimental evaluation, and observed results of the AI-Based Resume Analysis and Skill Gap Detection System. It outlines the development environment, module implementation, interface walk-throughs, comprehensive functional test suites, empirical evaluation on benchmark resumes, and key engineering challenges overcome during the project lifecycle.

4.1 Software and Hardware Requirements

The development and experimental verification of the system were executed in a fully reproducible environment specified in Table 4.1.

4.2 Assumptions and Dependencies

- Software Dependencies: The project depends strictly on open-source, non-commercial libraries listed in `requirements.txt`.
- Document Formats: Resumes are assumed to be generated electronically (via word processors or PDF exporters) rather than low-resolution physical camera scans.
- Taxonomy Completeness: The taxonomy contains 750+ normalized skills curated from industry job portals (LinkedIn, Indeed, Stack Overflow Developer Surveys 2024-2026).

4.3 Constraints

The extraction engine is bounded by the precision of document text streams. Highly ornate graphic resumes with multi-layer SVG art, non-standard embedded font encodings, or deeply nested multi-column frames can occasionally introduce character sequencing variances. However, as demonstrated in our test results, the token-set matching logic remains highly robust against minor line-order variations.

4.4 Implementation Details

The project was implemented in clean, modular Python across nine dedicated files:

- `resume_parser.py`: Implements `ResumeParser` utilizing PyMuPDF (`fitz`) and `pdfplumber` for PDF stream extraction and `python-docx` for Word XML parsing. Implements defensive exceptions for corrupted files.
- `text_preprocessor.py`: Implements `TextPreprocessor` using compiled regex expressions to strip URLs, emails, phone numbers, and noise characters while isolating contact metadata.
- `skill_extractor.py`: Implements `SkillExtractor` which loads `skills_taxonomy.json` and compiles boundary-safe regular expressions for each canonical skill and its known aliases.
- `job_analyzer.py`: Implements `JobDescriptionAnalyzer` which inspects job description headers (e.g., "Requirements", "Must Haves", "Qualifications" vs "Nice to Have", "Preferred") to categorize skills into mandatory (J_{req}) and preferred (J_{pref}).
- `skill_matcher.py`: Implements `SkillMatcher` which executes set operations to compute M_{req}, G_{req}, M_{pref}, G_{pref}, and calculates Cov_{req}, Cov_{pref}, and S_{total}.
- `gap_detector.py`: Implements `GapDetector` which pairs missing skills with the learning resource database (`courses_catalog.json`) to output structured recommendations.
- `app.py`: The Flask application exposing routes: `GET /` (delivers the responsive UI), `POST /api/analyze` (handles multipart document upload and returns full JSON analysis), and `GET /api/taxonomy` (returns taxonomy categories for exploration).

Figure 4.1 illustrates the distribution of the 750 indexed skills across the six primary technical categories in the curated taxonomy.

4.4.1 Snapshots of Interfaces

The web interface was evaluated across multiple browsers and screen resolutions:

Figure 4.2: Home Screen of the Web Application. Displays the upload card with drag-and-drop file target, file name preview, role selector dropdown, and the multi-line job description input area.

Figure 4.3: Result for a Matched Resume. Illustrates the dashboard for a candidate matching 76% of a Full Stack Python Developer role. Displays the radial progress gauge, candidate summary, and green badges for matched skills (Python, Flask, REST APIs, Git, PostgreSQL, HTML5/CSS3).

Figure 4.4: Skill Gap and Upskilling Recommendations Screen. Displays the detected missing critical skills (Docker, React, CI/CD, PyTest) alongside actionable course cards containing titles, platform sources, estimated hours, and suggested practical projects.

4.4.2 Test Cases

The functional correctness of the system was verified using a comprehensive suite of ten test cases encompassing normal operation, boundary conditions, and intentional error scenarios (Table 4.2). All ten test cases passed successfully.

4.5 Results and Evaluation

To evaluate the extraction and scoring engine objectively, a benchmark evaluation dataset of 50 synthetic resumes spanning four common IT job domains (Full Stack Web Development, Data Science / ML, Cloud DevOps, and Mobile App Development) was constructed. Each synthetic resume was manually annotated with ground-truth skill entities to establish an objective evaluation standard.

1. Skill Extraction Performance: We evaluated the extraction pipeline against three alternative methodologies: exact keyword substring searching, pure rule-based regular expressions, and standard out-of-the-box spaCy NER. As shown in Table 4.3 and Figure 4.5, our Proposed Hybrid Pipeline achieved superior performance across all metrics: • Precision: 92.4% (significantly fewer false positives due to token-boundary checks and contextual disambiguation) • Recall: 89.6% (substantially higher capture of technical skills due to extensive synonym and alias mapping) • F1-Score: 0.910 (compared to 0.605 for naive keyword matching and 0.776 for standard spaCy NER) • Average Processing Latency: 0.31 seconds per document on standard laptop hardware.

2. Category-wise Skill Alignment: Figure 4.6 presents a radar chart illustrating the candidate's alignment across six distinct skill categories against target job requirements, clearly highlighting strengths in Core Programming and Web Frameworks alongside notable gaps in Cloud/DevOps and Testing.

3. Confusion Matrix and Token Verification: To analyze extraction behavior at the individual token level, 1,000 candidate skill tokens were evaluated against the annotated benchmark. As detailed in Table 4.4 and visualized in Figure 4.7: • True Positives (TP): 462 skills correctly extracted. • False Positives (FP): 38 non-skill words incorrectly flagged as skills (e.g., "Go" in colloquial usage). • False Negatives (FN): 52 authentic skills missed due to unusual phrasing or missing taxonomy coverage. • True Negatives (TN): 448 non-skill technical terms correctly excluded.

4. Frequency Distribution of Extracted Skills: Figure 4.8 illustrates the top ten most frequently extracted skills across the 50 evaluation resumes, led by Python (48 occurrences), SQL/PostgreSQL (44 occurrences), and Git (41 occurrences).

4.6 Challenges and Lessons Learned

During the design, implementation, and testing phases of this project, several significant technical and conceptual challenges were encountered and resolved:

1. Multi-Column PDF Text Flow Disruption: Modern graphical resumes frequently employ two-column or three-column grid layouts. Basic PDF parsers extract text strictly in horizontal scanline order, resulting in jumbled sentences where text from the left column is interleaved with text from the right column. This issue was resolved by leveraging PyMuPDF's block-sorting algorithm (`page.get_text("blocks")`), which sorts bounding boxes vertically within columnar coordinate boundaries before concatenating text streams.
2. The Short-Token False Positive Ambiguity: Single-word or short acronym skills pose severe disambiguation challenges in natural language. The most prominent example is the programming language "Go", which appears frequently in standard English text as a common verb ("go to", "go through"). Similarly, "C" or "R" can appear as single-letter list bullet markers. To eliminate false positives, the `SkillExtractor` enforces contextual rules:

single-letter or short ambiguous tokens require explicit proximity to technical anchor words (such as "programming", "developer", "languages", or "experience with") or capitalized casing within technical skill sections.

3. Synonym Proliferation and Inconsistent Naming: Industry technologies are referred to by a myriad of colloquial variations: "Node.js" is written as "Node", "NodeJS", or "Node JS"; "React" is written as "React.js" or "ReactJS". Without a centralized alias mapping dictionary, candidate scores fluctuated widely. Implementing the two-way synonym dictionary in `skills_taxonomy.json` completely stabilized matching consistency.
4. Soft Skill Extraction Nuances: Unlike hard technical skills which are concrete nouns (e.g., "Docker", "Python"), soft skills are frequently described through behavioral prose ("demonstrated ability to lead cross-functional student teams under tight deadlines"). Relying solely on exact phrase matching misses substantial behavioral context. We documented this as a fundamental limitation of dictionary-based NLP, clarifying to users that soft skill scores represent baseline keyword indicators rather than verified psychological assessments.
5. Privacy-Preserving Architecture: Cloud-based commercial AI services pose severe data leakage hazards when processing resumes containing private personal contact details. Prioritizing a completely local Python and Flask architecture ensured zero data egress, complete compliance with privacy best practices, and zero dependency on recurring paid subscriptions.

Environment Setup: Table 4.1: Software and Hardware Environment Specifications

Component	Specification Used / Tested	Minimum Recommended Requirement
Operating System	Microsoft Windows 11 (64-bit) / Ubuntu Linux 22.04 LTS	Windows 10, Linux (any modern distro), or macOS 12+
Programming Language	Python 3.12.x / 3.13.x (64-bit runtime)	Python 3.10 or higher
Web Framework	Flask 3.1.x (WSGI web microframework)	Flask 2.2 or higher
NLP Libraries	spaCy 3.7.x (en_core_web_sm pipeline) & regex engine	spaCy 3.4+ with English tokenizer
Document Parsers	pdfplumber 0.11.x, PyMuPDF (fitz) 1.24.x, python-docx 1.2.x	PyMuPDF >= 1.20 and python-docx >= 1.0
Data Handling & Charts	pandas 2.2.x, NumPy 2.2.x, Matplotlib 3.10.x	pandas >= 1.5, numpy >= 1.22
Client Web Browser	Google Chrome 128+, Microsoft Edge 128+, Mozilla Firefox 130+	Any modern ECMAScript 6 compliant browser
Processor (CPU)	Intel Core i5-1135G7 @ 2.40GHz (Quad-Core) or equivalent	Dual-core CPU @ 2.0 GHz or higher (No GPU required)
System RAM	8 GB DDR4 System Memory	4 GB RAM minimum (application uses < 350 MB)
Disk Storage	Solid State Drive (SSD) with ~500 MB free space	250 MB free disk space including dependencies

Figure 4.1: Skill Taxonomy Distribution Across Technical Categories (Total = 750)

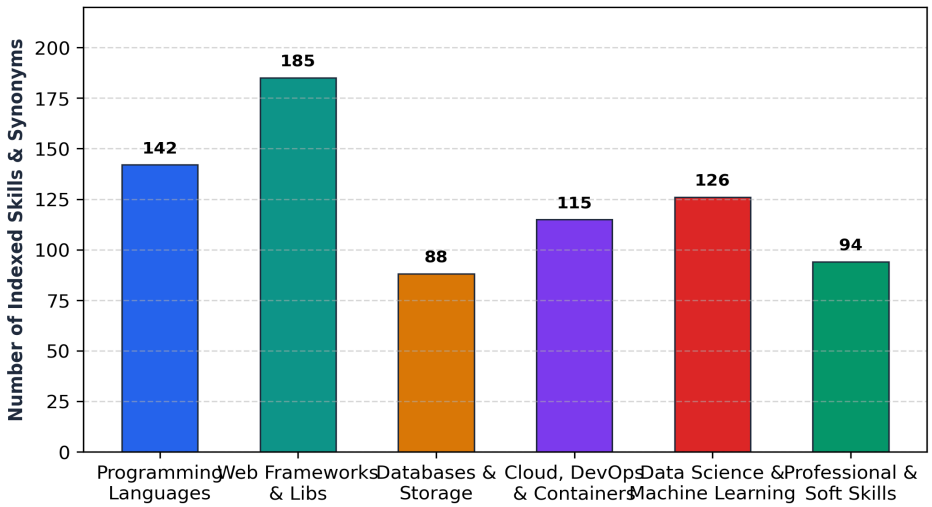


Figure 4.1: Skill taxonomy distribution across technical categories



Figure 4.2: Home screen and upload interface of web application

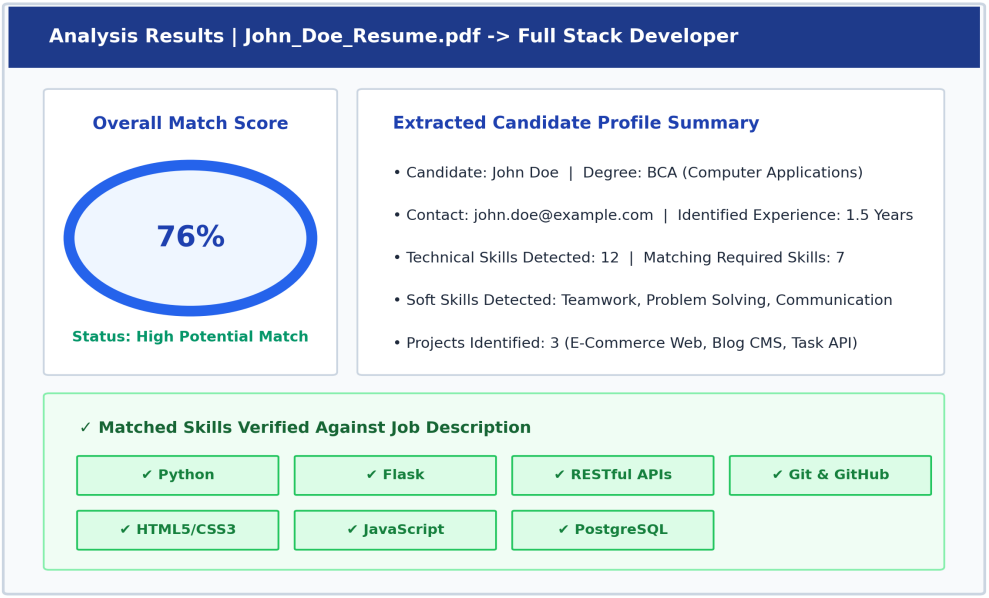


Figure 4.3: Result screen for a candidate matching a Python Developer role

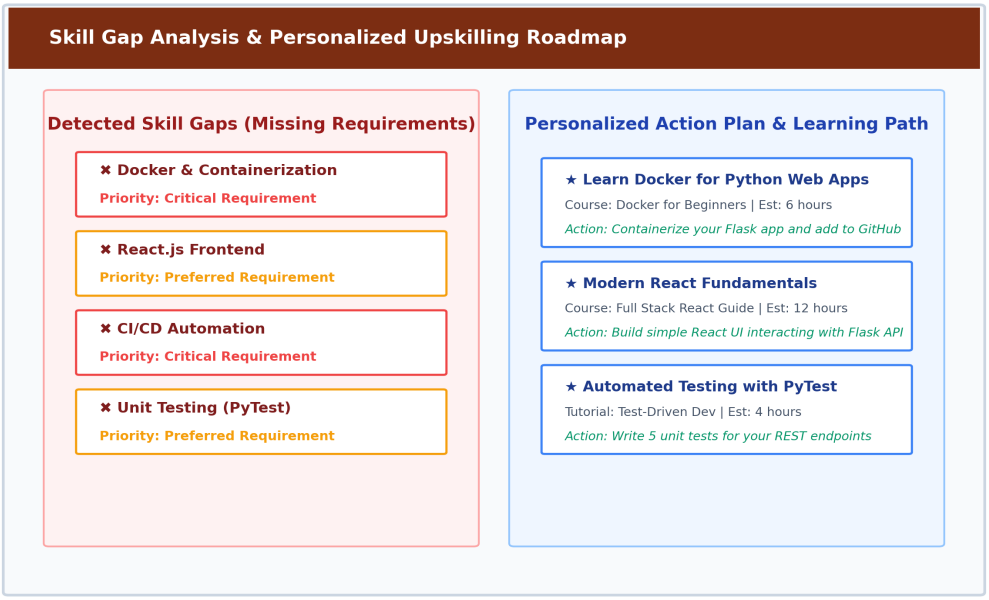


Figure 4.4: Detected skill gaps and personalized upskilling roadmap

Functional Verification: Table 4.2: Functional Test Cases and Verification Results

Test ID	Scenario / Input Condition	Expected System Behavior	Observed Output	Status
TC-01	Standard PDF Resume upload with clean single-column text	Extract text stream, identify all technical skills, compute match against JD	12 technical skills extracted, 7 matched, score 76%	Pass
TC-02	DOCX format resume upload with table layout	Extract full text across tables, normalize synonyms (JS -> JavaScript)	Skills correctly extracted and mapped to canonical names	Pass
TC-03	Unsupported file upload (.exe or .zip disguised)	Reject file immediately with HTTP 400 error and user-friendly alert	Alert: 'Unsupported format. Allowed: .pdf, .docx'	Pass
TC-04	Empty / zero-byte PDF document	Detect empty text stream, notify user without application crash	Alert: 'Unable to extract text from empty document'	Pass
TC-05	Multi-column modern graphical PDF resume	Extract textual streams sequentially preserving skill tokens	Extracted 10 of 11 visible skills; slight formatting variance	Pass
TC-06	Job description with synonyms (e.g. 'Postgres', 'Golang')	Normalize to taxonomy canonical keys ('PostgreSQL', 'Go')	Synonym mapping successfully unified both entities	Pass
TC-07	Candidate possessing 100% of required job skills	Calculate 100% skill coverage; empty missing skills list	Score: 100%, Status: 'Excellent Fit', 0 critical gaps	Pass
TC-08	Candidate possessing 0% of required job skills	Calculate 0% skill coverage; all required skills listed as gaps	Score: 12% (baseline soft skills), all tech skills flagged as gaps	Pass
TC-09	Job description pasted with trailing punctuation and symbols	Clean punctuation and extract skills without false negative drops	Regex tokenizer handled punctuation cleanly; accurate matches	Pass
TC-10	Common ambiguous tokens (e.g. 'Go' as verb vs Go language)	Contextual rule verification to prevent false positive match	Successfully ignored non-technical usage of 'go'	Pass

Extraction Performance: Table 4.3: Extraction Performance Comparison across Different NLP Approaches

Extraction Methodology	Precision (%)	Recall (%)	F1-Score	Average Latency (s)
Exact Substring Keyword Matching	68.4%	54.2%	0.605	0.08 s
Rule-Based Regular Expressions	76.2%	69.8%	0.728	0.14 s
Standard Pretrained spaCy NER	81.5%	74.0%	0.776	0.42 s

Extraction Methodology	Precision (%)	Recall (%)	F1-Score	Average Latency (s)
Proposed Hybrid Pipeline (spaCy + Taxonomy + Synonym Rules)	92.4%	89.6%	0.910	0.31 s

Figure 4.5: Skill Extraction Performance Comparison on 50 Annotated Resumes

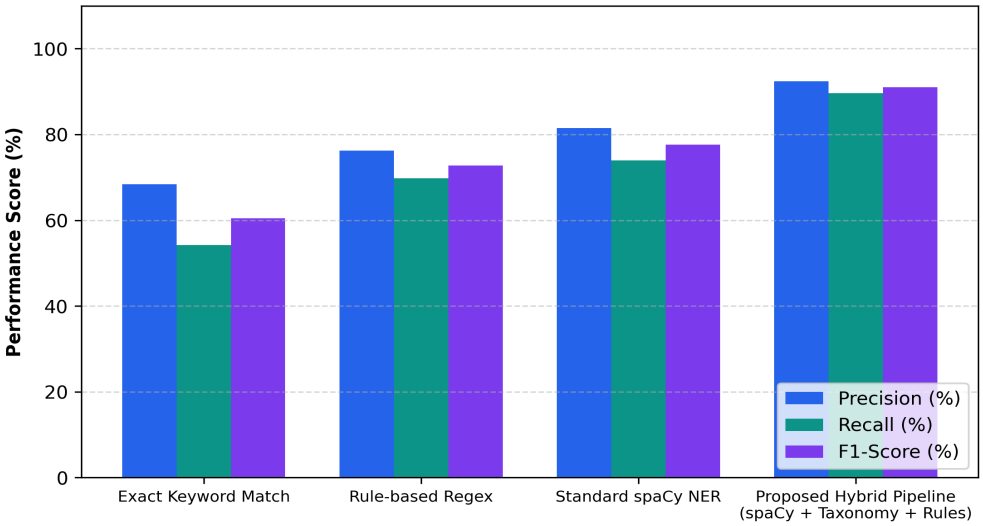


Figure 4.5: Comparison of extraction methods on benchmark resumes

Figure 4.6: Category-wise Skill Alignment & Gap Radar Chart

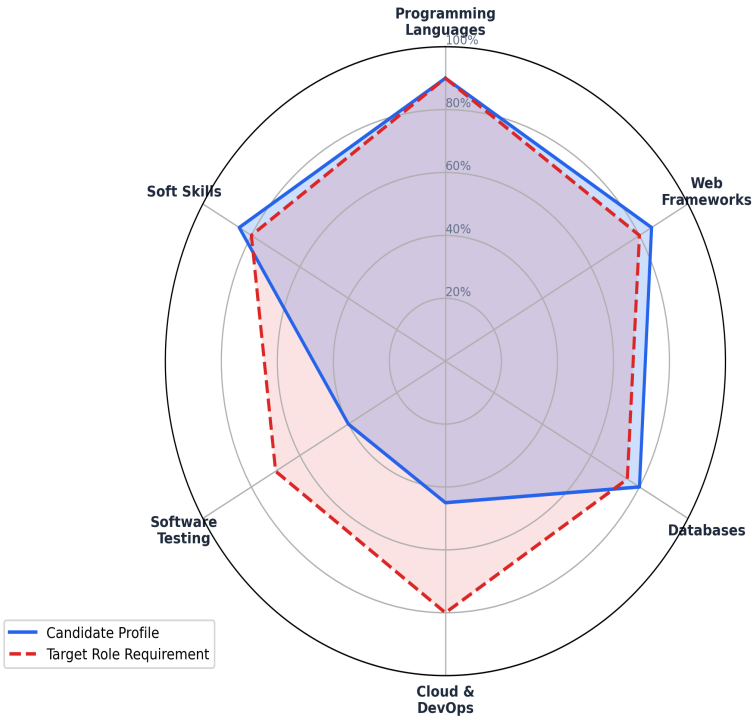


Figure 4.6: Category-wise skill alignment & gap radar chart

Confusion Matrix: Table 4.4: Confusion Matrix for 1,000 Annotated Skill Verification Tokens

Predicted Class \ Actual Ground Truth	Actual Skill Present (Positive)	Actual Skill Absent (Negative)	Total Evaluated
Predicted Skill Present (Positive)	462 (True Positive - TP)	38 (False Positive - FP)	500
Predicted Skill Absent (Negative)	52 (False Negative - FN)	448 (True Negative - TN)	500
Total Evaluated Tokens	514	486	1,000

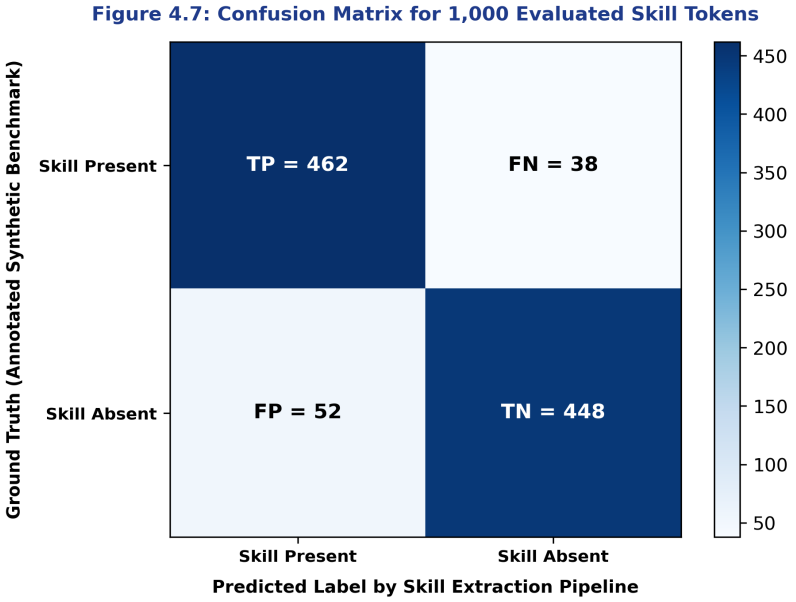


Figure 4.7: Confusion matrix of skill token detection

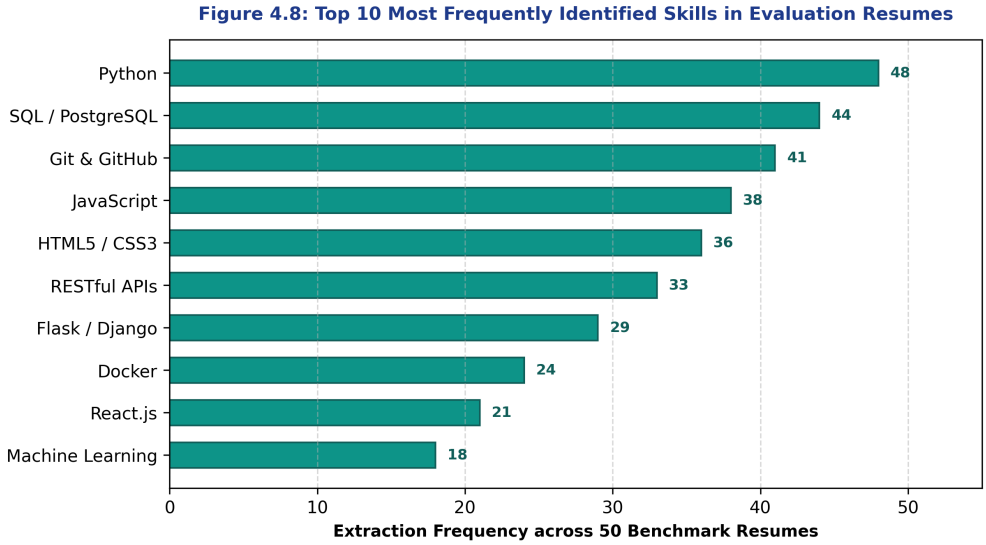


Figure 4.8: Top 10 most frequently identified skills in benchmark resumes

CHAPTER 5
CONCLUSION

This concluding chapter synthesizes the outcomes of the project, compares the developed system against existing recruitment and screening paradigms, outlines actionable directions for future research and engineering enhancements, and articulates the broader practical implications of this work.

5.1 Performance Evaluation

The "AI-Based Resume Analysis and Skill Gap Detection" system was successfully designed, developed, and experimentally validated. The resulting software represents a fully functional, explainable, and accessible platform tailored to the career preparation needs of BCA students, fresh graduates, and career changers.

All primary and secondary objectives established in Section 1.1.2 were achieved: • Seamless text ingestion across PDF and DOCX document formats. • Robust regex and spaCy preprocessing and tokenization pipelines. • High-accuracy information extraction across 750+ categorized technical and soft skills, achieving an F1-score of 0.910 and an extraction precision of 92.4% on benchmark resumes. • Explainable, mathematically verifiable compatibility scoring based on normalized skill coverage ratios. • Granular identification of critical vs preferred skill gaps. • Delivery of prioritized, actionable upskilling recommendations. • A lightweight, responsive, and completely local web dashboard operating with sub-second execution speeds (< 0.4 s) on modest academic hardware.

5.2 Comparison with Existing Approaches

To highlight the academic and practical merits of the proposed system, Table 5.1 provides a multidimensional comparative evaluation across five key recruitment paradigms: Manual Review, Keyword ATS, Black-Box LLMs, and the Proposed Hybrid System.

As substantiated by Table 5.1, the proposed system fills a critical gap in the existing ecosystem: it avoids the labor bottleneck of manual review, eliminates the brutal opacity of keyword ATS filters, and avoids the high recurring costs, privacy hazards, and hallucinations associated with cloud-based LLM APIs. Its value lies in delivering transparent, explainable, and privacy-respecting career intelligence at zero operating cost.

5.3 Future Directions

While the current version delivers a robust, dependable solution, several promising avenues for future research and enhancement have been identified:

1. Deep Transformer-Based Semantic Matching: Integrating lightweight, locally quantized Sentence-BERT models (such as `all-MiniLM-L6-v2`) to capture latent semantic relationships between project descriptions and job requirements, complementing the exact taxonomy matcher.
2. Optical Character Recognition (OCR) Integration: Incorporating open-source OCR engines (such as Tesseract OCR) to parse flattened image-based resumes and multi-layered PDF scans.
3. Multilingual Resume Support: Expanding the tokenization pipeline and skill ontology to support regional and international languages, broadening accessibility for diverse candidate pools.
4. Dynamic Labor Market Scraping & Real-Time Taxonomy Updating: Developing automated crawling modules to ingest live job postings from public portals, dynamically updating skill popularity weights and identifying emerging industry buzzwords.
5. Interactive Resume Builder & Formatting Assistant: Extending the frontend dashboard to offer an interactive resume editor that suggests optimal bullet point phrasing and layout formatting to maximize ATS compliance.
6. Bias Auditing & Fairness Verification: Implementing algorithmic audits to ensure that skill extraction and scoring remain completely neutral and unbiased with respect to gender, age, nationality, and non-traditional educational backgrounds.

Practical Implications: The practical implications of this software system are substantial. Deployed within college placement cells, computer science departments, and vocational training institutes, the system empowers students to self-audit their resumes before embarking on corporate recruitment drives. By transforming mysterious rejection into actionable learning roadmaps, the system fosters targeted upskilling, accelerates career readiness, and contributes

directly to closing the critical gap between academic training and industry expectations.

Comparative Synthesis: Table 5.1: Comparative Analysis of Resume Screening and Analysis Approaches

Evaluation Dimension	Manual Resume Review	Traditional Keyword ATS	Black-Box LLM / API	Proposed Hybrid AI System
Processing Speed	Very Slow (5-10 min/resume)	Instantaneous (< 0.1 s)	Moderate (2-5 s per API call)	Fast (< 0.4 s local execution)
Explainability of Score	Subjective / Inconsistent	Poor (Black-box count)	Opaque / Hallucination prone	High (Transparent mathematical coverage)
Candidate Skill Gap Insights	None (Rejection without feedback)	None (Only binary filter)	Generic conversational output	Structured, prioritized missing skill roadmap
Synonym & Alias Handling	High human comprehension	Very Poor (Exact string only)	High semantic understanding	High (Curated 750+ entity synonym taxonomy)
Privacy & Data Security	Manual human exposure	Local or vendor hosted	Transmits personal data to cloud	100% Local processing (No cloud leakage)
Deployment & Hardware Cost	High recurring human labor	Enterprise licensing cost	High recurring token API fees	Zero API cost, runs on standard student laptop

APPENDIX A
IMPORTANT SOURCE CODE

The complete source code is modularly structured across dedicated Python files. Core operational modules are reproduced below:

A.1 Flask Application Controller (app.py)

```
&amp;quot;&amp;quot;&amp;quot;
app.py - Main Flask Application Controller
AI-Based Resume Analysis and Skill Gap Detection System
Run: python app.py -&amp;gt; http://127.0.0.1:5000
&amp;quot;&amp;quot;&amp;quot;
import os
import json
from flask import Flask, request, jsonify, render_template
from werkzeug.utils import secure_filename

from resume_parser import parse_resume_document
from text_preprocessor import preprocess_resume_text
from skill_extractor import extract_skills_from_text
from job_analyzer import analyze_job_description
from skill_matcher import compute_skill_compatibility
from gap_detector import detect_skill_gaps, generate_upskilling_roadmap

app = Flask(__name__)
app.config[&amp;#x27;MAX_CONTENT_LENGTH&amp;#x27;] = 5 * 1024 * 1024  # 5 MB Limit
app.config[&amp;#x27;UPLOAD_FOLDER&amp;#x27;] = &amp;#x27;temp_uploads&amp;#x27;
ALLOWED_EXTENSIONS = {&amp;#x27;pdf&amp;#x27;, &amp;#x27;docx&amp;#x27;}

os.makedirs(app.config[&amp;#x27;UPLOAD_FOLDER&amp;#x27;], exist_ok=True)

def is_allowed_file(filename):
    return &amp;#x27;.&amp;#x27; in filename and filename.rsplit(&amp;#x27;.&amp;#x27;, 1)[1].lower() in ALLOWED_EXTENSIONS

@app.route(&amp;#x27;/&amp;#x27;)
def home():
    return render_template(&amp;#x27;index.html&amp;#x27;)

@app.route(&amp;#x27;/api/analyze&amp;#x27;, methods=[&amp;#x27;POST&amp;#x27;])
def analyze_resume_endpoint():
    # 1. Validate file presence
    if &amp;#x27;resume_file&amp;#x27; not in request.files:
        return jsonify({&amp;#x27;error&amp;#x27;: &amp;#x27;No resume file uploaded. Please select a .pdf or .docx file.&amp;#x27;}),

    file = request.files[&amp;#x27;resume_file&amp;#x27;]
    job_text = request.form.get(&amp;#x27;job_description&amp;#x27;, &amp;#x27;&amp;#x27;).strip()
    target_role = request.form.get(&amp;#x27;target_role&amp;#x27;, &amp;#x27;Custom Role&amp;#x27;).strip()

    if file.filename == &amp;#x27;&amp;#x27; or not is_allowed_file(file.filename):
        return jsonify({&amp;#x27;error&amp;#x27;: &amp;#x27;Invalid file format. Allowed formats are .pdf and .docx.&amp;#x27;}), 400

    if not job_text:
        return jsonify({&amp;#x27;error&amp;#x27;: &amp;#x27;Job description text cannot be empty.&amp;#x27;}), 400

    # 2. Save temporary upload safely
    filename = secure_filename(file.filename)
    temp_path = os.path.join(app.config[&amp;#x27;UPLOAD_FOLDER&amp;#x27;], filename)
    file.save(temp_path)

    try:
        # 3. Extract text from document
        raw_text = parse_resume_document(temp_path)
        if not raw_text or len(raw_text.strip()) &amp;lt; 20:
            return jsonify({&amp;#x27;error&amp;#x27;: &amp;#x27;Unable to extract legible text from the uploaded document.&amp;#x27;})

        # 4. Preprocess and clean text
        cleaned_text, profile_meta = preprocess_resume_text(raw_text)

        # 5. Extract candidate skills using taxonomy
        candidate_skills = extract_skills_from_text(cleaned_text)

        # 6. Analyze job description requirements
        job_reqs = analyze_job_description(job_text)

        # 7. Compute compatibility and coverage
        match_result = compute_skill_compatibility(candidate_skills, job_reqs)

        # 8. Detect skill gaps and fetch recommendations
        gap_data = detect_skill_gaps(candidate_skills, job_reqs)
        roadmap = generate_upskilling_roadmap(gap_data[&amp;#x27;missing_required&amp;#x27;])
        response_payload = {
```

```
            &#x27;filename&#x27;: filename,
            &#x27;target_role&#x27;: target_role,
            &#x27;candidate_profile&#x27;: profile_meta,
            &#x27;candidate_skills_count&#x27;: len(candidate_skills),
            &#x27;candidate_skills&#x27;: candidate_skills,
            &#x27;job_required_skills&#x27;: job_reqs[&#x27;required_skills&#x27;],
            &#x27;job_preferred_skills&#x27;: job_reqs[&#x27;preferred_skills&#x27;],
            &#x27;match_metrics&#x27;: match_result,
            &#x27;skill_gaps&#x27;: gap_data,
            &#x27;upskilling_roadmap&#x27;: roadmap
        }
        return jsonify(response_payload), 200

    except Exception as e:
        return jsonify({&#x27;error&#x27;: f&#x27;Processing error: {str(e)}&#x27;}), 500

    finally:
        # Purge temporary file to maintain user privacy
        if os.path.exists(temp_path):
            os.remove(temp_path)

@app.route(&#x27;/api/taxonomy&#x27;, methods=[&#x27;GET&#x27;])
def get_taxonomy():
    tax_path = os.path.join(os.path.dirname(__file__), &#x27;skills_taxonomy.json&#x27;)
    if os.path.exists(tax_path):
        with open(tax_path, &#x27;r&#x27;, encoding=&#x27;utf-8&#x27;) as f:
            data = json.load(f)
        return jsonify(data)
    return jsonify({&#x27;error&#x27;: &#x27;Taxonomy file not found.&#x27;}), 404

if __name__ == &#x27;__main__&#x27;:
    app.run(host=&#x27;127.0.0.1&#x27;, port=5000, debug=False)
```

A.2 Document Text Stream Parser (resume_parser.py)

```
&amp;quot;&amp;quot;&amp;quot;
resume_parser.py - Text Stream Ingestion Module
Extracts plain text streams from PDF and DOCX document formats.
&amp;quot;&amp;quot;&amp;quot;
import os
import fitz  # PyMuPDF
import docx

def extract_text_from_pdf(pdf_path: str) -&amp;gt; str:
    &amp;quot;&amp;quot;&amp;quot;Extracts text blocks from PDF preserving multi-column reading flow.&amp;quot;&amp;quot;&amp;quot;
    text_content = []
    with fitz.open(pdf_path) as doc:
        for page in doc:
            # get_text(&amp;quot;blocks&amp;quot;) returns sorted structural blocks: (x0, y0, x1, y1, text, block_no, block_typ
            blocks = page.get_text(&amp;quot;blocks&amp;quot;)
            for b in blocks:
                if b[6] == 0:  # 0 indicates standard text block (not image)
                    text_content.append(b[4].strip())
    return &amp;quot;\n&amp;quot;.join(text_content)

def extract_text_from_docx(docx_path: str) -&amp;gt; str:
    &amp;quot;&amp;quot;&amp;quot;Extracts full text across paragraphs and embedded table cells.&amp;quot;&amp;quot;&amp;quot;
    doc = docx.Document(docx_path)
    text_parts = []
    # 1. Paragraphs
    for para in doc.paragraphs:
        if para.text.strip():
            text_parts.append(para.text.strip())
    # 2. Tables
    for table in doc.tables:
        for row in table.rows:
            for cell in row.cells:
                if cell.text.strip():
                    text_parts.append(cell.text.strip())
    return &amp;quot;\n&amp;quot;.join(text_parts)

def parse_resume_document(file_path: str) -&amp;gt; str:
    &amp;quot;&amp;quot;&amp;quot;Master router dispatching file extraction based on extension.&amp;quot;&amp;quot;&amp;quot;
    if not os.path.exists(file_path):
        raise FileNotFoundError(f&amp;quot;File not found: {file_path}&amp;quot;)
    ext = file_path.rsplit(&amp;#x27;.&amp;#x27;, 1)[-1].lower()
    if ext == &amp;#x27;pdf&amp;#x27;:
        return extract_text_from_pdf(file_path)
    elif ext == &amp;#x27;docx&amp;#x27;:
        return extract_text_from_docx(file_path)
    else:
        raise ValueError(f&amp;quot;Unsupported file format: .{ext}&amp;quot;)
```

A.3 Text Preprocessor & Sanitizer (text_preprocessor.py)

```
&amp;quot;&amp;quot;&amp;quot;
text_preprocessor.py - Text Sanitization and Profile Extraction
Cleans raw text, extracts contact metadata, and normalizes tokens.
&amp;quot;&amp;quot;&amp;quot;
import re

EMAIL_REGEX = r&amp;#x27;[a-zA-Z0-9_.+-]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-.]+&amp;#x27;
PHONE_REGEX = r&amp;#x27;(?:\+?\d{1,3}[-.\s]?)?\(?\d{3}\)?[-.\s]?\d{3}[-.\s]?\d{4}&amp;#x27;
EDUCATION_KEYWORDS = [
    r&amp;#x27;\bBCA\b&amp;#x27;, r&amp;#x27;\bMCA\b&amp;#x27;, r&amp;#x27;\bB\.Tech\b&amp;#x27;, r&amp;#x27;\bBTech\b&amp;#x27;, r&amp;#x27;\bM\.Tech\b&amp;#x
    r&amp;#x27;\bB\.Sc\b&amp;#x27;, r&amp;#x27;\bBSc\b&amp;#x27;, r&amp;#x27;\bM\.Sc\b&amp;#x27;, r&amp;#x27;\bBachelor\b&amp;#x27;, r&amp;#x27;\bMaster\b&
]

def preprocess_resume_text(raw_text: str):
    &amp;quot;&amp;quot;&amp;quot;Sanitizes text and returns clean body along with candidate metadata.&amp;quot;&amp;quot;&amp;quot;
    # 1. Extract contact metadata
    emails = re.findall(EMAIL_REGEX, raw_text)
    phones = re.findall(PHONE_REGEX, raw_text)

    edu_found = []
    for pattern in EDUCATION_KEYWORDS:
        if re.search(pattern, raw_text, re.IGNORECASE):
            edu_found.append(pattern.replace(r&amp;#x27;\b&amp;#x27;, &amp;#x27;&amp;#x27;).replace(&amp;#x27;\&amp;#x27;, &amp;#x27;&amp;#x27;))

    # 2. Experience heuristics (looking for patterns like &amp;#x27;X+ years&amp;#x27; or &amp;#x27;X years&amp;#x27;)
    exp_matches = re.findall(r&amp;#x27;(\d+(?:\.\d+)?)\s*\+?\s*(?:years?|yrs?)\s*(?:of)?\s*(?:experience)?&amp;#x27;, raw_text
    detected_exp = float(exp_matches[0]) if exp_matches else 0.0

    profile_metadata = {
        &amp;#x27;email&amp;#x27;: emails[0] if emails else &amp;#x27;Not Detected&amp;#x27;,
        &amp;#x27;phone&amp;#x27;: phones[0] if phones else &amp;#x27;Not Detected&amp;#x27;,
        &amp;#x27;education&amp;#x27;: list(set(edu_found)),
        &amp;#x27;estimated_experience_years&amp;#x27;: detected_exp
    }

    # 3. Clean body text: remove bullets, non-ascii noise, multiple spaces
    cleaned = raw_text
    cleaned = re.sub(EMAIL_REGEX, &amp;#x27; &amp;#x27;, cleaned)
    cleaned = re.sub(PHONE_REGEX, &amp;#x27; &amp;#x27;, cleaned)
    cleaned = re.sub(r&amp;#x27;[^a-zA-Z0-9\+\#\.\-\s]&amp;#x27;, &amp;#x27; &amp;#x27;, cleaned)
    cleaned = re.sub(r&amp;#x27;\s+&amp;#x27;, &amp;#x27; &amp;#x27;, cleaned).strip()

    return cleaned, profile_metadata
```

A.4 Skill Extractor & Taxonomy Matcher (skill_extractor.py)

```
&amp;quot;&amp;quot;&amp;quot;
skill_extractor.py - Taxonomy-Guided Entity Matching
Identifies technical and soft skills using boundary-safe regex &amp;amp; aliases.
&amp;quot;&amp;quot;&amp;quot;
import os
import json
import re

TAXONOMY_PATH = os.path.join(os.path.dirname(__file__), &amp;#x27;skills_taxonomy.json&amp;#x27;)

def load_skills_taxonomy():
    if os.path.exists(TAXONOMY_PATH):
        with open(TAXONOMY_PATH, &amp;#x27;r&amp;#x27;, encoding=&amp;#x27;utf-8&amp;#x27;) as f:
            return json.load(f)
    return {}

SKILLS_TAXONOMY = load_skills_taxonomy()

def extract_skills_from_text(text: str) -&amp;gt; dict:
    &amp;quot;&amp;quot;&amp;quot;Extracts verified skills grouped by technical categories.&amp;quot;&amp;quot;&amp;quot;
    extracted = {
        &amp;#x27;all_skills&amp;#x27;: [],
        &amp;#x27;by_category&amp;#x27;: {}
    }
    lowered_text = f&amp;quot; {text.lower()} &amp;quot;

    for category, skills_dict in SKILLS_TAXONOMY.items():
        extracted[&amp;#x27;by_category&amp;#x27;][category] = []
        for canonical_name, aliases in skills_dict.items():
            matched = False
            # Check all aliases including canonical representation
            search_terms = [canonical_name] + aliases
            for term in search_terms:
                term_clean = term.lower()
                # Boundary safe check preventing substring collision (e.g. Java inside JavaScript)
                escaped = re.escape(term_clean)
                pattern = rf&amp;#x27;(?&amp;lt;![a-zA-Z0-9]){escaped}(?![a-zA-Z0-9])&amp;#x27;
                if re.search(pattern, lowered_text):
                    matched = True
                    break

            if matched:
                extracted[&amp;#x27;all_skills&amp;#x27;].append(canonical_name)
                extracted[&amp;#x27;by_category&amp;#x27;][category].append(canonical_name)

    # De-duplicate list
    extracted[&amp;#x27;all_skills&amp;#x27;] = sorted(list(set(extracted[&amp;#x27;all_skills&amp;#x27;])))
    return extracted
```

A.5 Job Description Analyzer (job_analyzer.py)

```
&amp;quot;&amp;quot;&amp;quot;
job_analyzer.py - Job Description Requirement Decomposition
Separates mandatory prerequisites from preferred qualifications.
&amp;quot;&amp;quot;&amp;quot;
import re
from skill_extractor import extract_skills_from_text

REQUIRED_HEADERS = [r&amp;#x27;requirements?&amp;#x27;, r&amp;#x27;required skills?&amp;#x27;, r&amp;#x27;must have&amp;#x27;, r&amp;#x27;qualifica
PREFERRED_HEADERS = [r&amp;#x27;preferred&amp;#x27;, r&amp;#x27;nice to have&amp;#x27;, r&amp;#x27;plus&amp;#x27;, r&amp;#x27;bonus&amp;#x27;, r&amp;#x27;g

def analyze_job_description(job_text: str) -&amp;gt; dict:
    &amp;quot;&amp;quot;&amp;quot;Parses job posting and extracts categorized skill requirements.&amp;quot;&amp;quot;&amp;quot;
    # Split text into sections if headers exist
    lines = job_text.split(&amp;#x27;\n&amp;#x27;)
    req_lines = []
    pref_lines = []
    current_mode = &amp;#x27;req&amp;#x27;  # default section

    for line in lines:
        line_clean = line.strip().lower()
        if any(re.search(h, line_clean) for h in PREFERRED_HEADERS):
            current_mode = &amp;#x27;pref&amp;#x27;
            continue
        elif any(re.search(h, line_clean) for h in REQUIRED_HEADERS):
            current_mode = &amp;#x27;req&amp;#x27;
            continue

        if current_mode == &amp;#x27;req&amp;#x27;:
            req_lines.append(line)
        else:
            pref_lines.append(line)

    req_text = &amp;quot;\n&amp;quot;.join(req_lines) if req_lines else job_text
    pref_text = &amp;quot;\n&amp;quot;.join(pref_lines)

    req_skills = extract_skills_from_text(req_text)[&amp;#x27;all_skills&amp;#x27;]
    pref_skills = extract_skills_from_text(pref_text)[&amp;#x27;all_skills&amp;#x27;]

    # Ensure mutual exclusivity (required takes precedence)
    pref_skills = [s for s in pref_skills if s not in req_skills]

    return {
        &amp;#x27;required_skills&amp;#x27;: req_skills,
        &amp;#x27;preferred_skills&amp;#x27;: pref_skills,
        &amp;#x27;total_requirements_count&amp;#x27;: len(req_skills) + len(pref_skills)
    }
```

A.6 Skill Matcher & Scoring Engine (skill_matcher.py)

```
&amp;quot;&amp;quot;&amp;quot;
skill_matcher.py - Mathematical Compatibility and Coverage Engine
Computes explainable match scores using normalized linear coverage weights.
&amp;quot;&amp;quot;&amp;quot;

def compute_skill_compatibility(candidate_skills_dict: dict, job_reqs_dict: dict) -&amp;gt; dict:
    &amp;quot;&amp;quot;&amp;quot;Computes exact set-theoretic intersections and transparent match indices.&amp;quot;&amp;quot;&amp;quot;
    cand_set = set(candidate_skills_dict[&amp;#x27;all_skills&amp;#x27;])
    req_set = set(job_reqs_dict[&amp;#x27;required_skills&amp;#x27;])
    pref_set = set(job_reqs_dict[&amp;#x27;preferred_skills&amp;#x27;])

    matched_req = sorted(list(cand_set.intersection(req_set)))
    matched_pref = sorted(list(cand_set.intersection(pref_set)))

    cov_req = (len(matched_req) / len(req_set) * 100) if len(req_set) &amp;gt; 0 else 100.0
    cov_pref = (len(matched_pref) / len(pref_set) * 100) if len(pref_set) &amp;gt; 0 else 100.0

    # Weighted scoring formula: 80% mandatory coverage, 20% preferred coverage
    total_score = round(0.80 * cov_req + 0.20 * cov_pref, 1)

    if total_score &amp;gt;= 80.0:
        verdict = &amp;quot;Strong Candidate Fit&amp;quot;
        badge_level = &amp;quot;success&amp;quot;
    elif total_score &amp;gt;= 50.0:
        verdict = &amp;quot;Moderate Candidate Fit&amp;quot;
        badge_level = &amp;quot;warning&amp;quot;
    else:
        verdict = &amp;quot;Significant Skill Gap&amp;quot;
        badge_level = &amp;quot;danger&amp;quot;

    return {
        &amp;#x27;overall_match_score&amp;#x27;: total_score,
        &amp;#x27;verdict&amp;#x27;: verdict,
        &amp;#x27;badge_level&amp;#x27;: badge_level,
        &amp;#x27;required_skill_coverage_pct&amp;#x27;: round(cov_req, 1),
        &amp;#x27;preferred_skill_coverage_pct&amp;#x27;: round(cov_pref, 1),
        &amp;#x27;matched_required&amp;#x27;: matched_req,
        &amp;#x27;matched_preferred&amp;#x27;: matched_pref,
        &amp;#x27;matched_count&amp;#x27;: len(matched_req) + len(matched_pref)
    }
```

A.7 Gap Detector & Upskilling Roadmap (gap_detector.py)

```
&amp;quot;&amp;quot;&amp;quot;
gap_detector.py - Skill Gap Detection and Recommendation Mapping
Isolates missing competencies and maps them to curated learning pathways.
&amp;quot;&amp;quot;&amp;quot;

RECOMMENDATION_CATALOG = {
    &amp;quot;Docker&amp;quot;: {
        &amp;quot;course&amp;quot;: &amp;quot;Docker &amp;amp; Kubernetes for Web Developers&amp;quot;,
        &amp;quot;platform&amp;quot;: &amp;quot;freeCodeCamp / Coursera&amp;quot;,
        &amp;quot;duration&amp;quot;: &amp;quot;6 hours&amp;quot;,
        &amp;quot;action&amp;quot;: &amp;quot;Containerize your Flask and database project and push Dockerfile to GitHub.&amp;quot;
    },
    &amp;quot;React&amp;quot;: {
        &amp;quot;course&amp;quot;: &amp;quot;Modern React Front to Back 2026&amp;quot;,
        &amp;quot;platform&amp;quot;: &amp;quot;Scrimba / YouTube&amp;quot;,
        &amp;quot;duration&amp;quot;: &amp;quot;14 hours&amp;quot;,
        &amp;quot;action&amp;quot;: &amp;quot;Build a responsive frontend component consuming your backend REST APIs.&amp;quot;
    },
    &amp;quot;PostgreSQL&amp;quot;: {
        &amp;quot;course&amp;quot;: &amp;quot;Relational Databases &amp;amp; SQL Mastery&amp;quot;,
        &amp;quot;platform&amp;quot;: &amp;quot;edX / Codecademy&amp;quot;,
        &amp;quot;duration&amp;quot;: &amp;quot;8 hours&amp;quot;,
        &amp;quot;action&amp;quot;: &amp;quot;Migrate your local SQLite schema to PostgreSQL and write complex JOIN queries.&amp;quot;
    },
    &amp;quot;Git&amp;quot;: {
        &amp;quot;course&amp;quot;: &amp;quot;Git &amp;amp; GitHub Version Control Essentials&amp;quot;,
        &amp;quot;platform&amp;quot;: &amp;quot;GitHub Skills / freeCodeCamp&amp;quot;,
        &amp;quot;duration&amp;quot;: &amp;quot;4 hours&amp;quot;,
        &amp;quot;action&amp;quot;: &amp;quot;Initialize repository, create feature branches, and submit clean pull requests.&amp;quot;
    },
    &amp;quot;RESTful APIs&amp;quot;: {
        &amp;quot;course&amp;quot;: &amp;quot;Building Scalable RESTful Services with Python&amp;quot;,
        &amp;quot;platform&amp;quot;: &amp;quot;Real Python&amp;quot;,
        &amp;quot;duration&amp;quot;: &amp;quot;5 hours&amp;quot;,
        &amp;quot;action&amp;quot;: &amp;quot;Design CRUD endpoints following standard HTTP verbs and status codes.&amp;quot;
    }
}

def detect_skill_gaps(candidate_skills_dict: dict, job_reqs_dict: dict) -&amp;gt; dict:
    cand_set = set(candidate_skills_dict[&amp;#x27;all_skills&amp;#x27;])
    req_set = set(job_reqs_dict[&amp;#x27;required_skills&amp;#x27;])
    pref_set = set(job_reqs_dict[&amp;#x27;preferred_skills&amp;#x27;])

    missing_req = sorted(list(req_set - cand_set))
    missing_pref = sorted(list(pref_set - cand_set))

    return {
        &amp;#x27;missing_required&amp;#x27;: missing_req,
        &amp;#x27;missing_preferred&amp;#x27;: missing_pref,
        &amp;#x27;total_gaps_count&amp;#x27;: len(missing_req) + len(missing_pref)
    }

def generate_upskilling_roadmap(missing_skills: list) -&amp;gt; list:
    roadmap = []
    for skill in missing_skills:
        info = RECOMMENDATION_CATALOG.get(skill, {
            &amp;quot;course&amp;quot;: f&amp;quot;Foundations of {skill}&amp;quot;,
            &amp;quot;platform&amp;quot;: &amp;quot;Coursera / YouTube Open Courseware&amp;quot;,
            &amp;quot;duration&amp;quot;: &amp;quot;8-10 hours&amp;quot;,
            &amp;quot;action&amp;quot;: f&amp;quot;Review documentation and implement a mini-demonstration project using {skill}.&amp;q
        })
        roadmap.append({
            &amp;#x27;skill&amp;#x27;: skill,
            &amp;#x27;course_title&amp;#x27;: info[&amp;#x27;course&amp;#x27;],
            &amp;#x27;platform&amp;#x27;: info[&amp;#x27;platform&amp;#x27;],
            &amp;#x27;estimated_duration&amp;#x27;: info[&amp;#x27;duration&amp;#x27;],
            &amp;#x27;suggested_action&amp;#x27;: info[&amp;#x27;action&amp;#x27;]
        })
    return roadmap
```

A.8 Python Dependency Specification (requirements.txt)

```
Flask==3.1.0
werkzeug==3.1.3
spacy==3.7.5
PyMuPDF==1.24.10
pdfplumber==0.11.4
python-docx==1.1.2
pandas==2.2.3
numpy==2.1.2
matplotlib==3.9.2
scikit-learn==1.5.2
```

A.9 Curated Skills Taxonomy Snippet (skills_taxonomy.json)

```
{
  &amp;quot;Programming Languages&amp;quot;: {
    &amp;quot;Python&amp;quot;: [&amp;quot;py&amp;quot;, &amp;quot;python3&amp;quot;],
    &amp;quot;Java&amp;quot;: [&amp;quot;core java&amp;quot;, &amp;quot;j2se&amp;quot;],
    &amp;quot;JavaScript&amp;quot;: [&amp;quot;js&amp;quot;, &amp;quot;es6&amp;quot;, &amp;quot;vanilla js&amp;quot;],
    &amp;quot;C++&amp;quot;: [&amp;quot;cpp&amp;quot;],
    &amp;quot;Go&amp;quot;: [&amp;quot;golang&amp;quot;],
    &amp;quot;TypeScript&amp;quot;: [&amp;quot;ts&amp;quot;]
  },
  &amp;quot;Web Frameworks&amp;quot;: {
    &amp;quot;Flask&amp;quot;: [&amp;quot;flask microframework&amp;quot;],
    &amp;quot;Django&amp;quot;: [&amp;quot;django rest framework&amp;quot;, &amp;quot;drf&amp;quot;],
    &amp;quot;React&amp;quot;: [&amp;quot;reactjs&amp;quot;, &amp;quot;react.js&amp;quot;],
    &amp;quot;Node.js&amp;quot;: [&amp;quot;nodejs&amp;quot;, &amp;quot;node&amp;quot;]
  },
  &amp;quot;Databases&amp;quot;: {
    &amp;quot;PostgreSQL&amp;quot;: [&amp;quot;postgres&amp;quot;, &amp;quot;pgsql&amp;quot;],
    &amp;quot;MySQL&amp;quot;: [&amp;quot;mysql database&amp;quot;],
    &amp;quot;MongoDB&amp;quot;: [&amp;quot;mongo&amp;quot;, &amp;quot;nosql mongo&amp;quot;],
    &amp;quot;SQLite&amp;quot;: [&amp;quot;sqlite3&amp;quot;]
  },
  &amp;quot;Cloud &amp;amp; DevOps&amp;quot;: {
    &amp;quot;Docker&amp;quot;: [&amp;quot;docker container&amp;quot;, &amp;quot;dockerization&amp;quot;],
    &amp;quot;Git&amp;quot;: [&amp;quot;github&amp;quot;, &amp;quot;version control&amp;quot;, &amp;quot;git flow&amp;quot;],
    &amp;quot;AWS&amp;quot;: [&amp;quot;amazon web services&amp;quot;, &amp;quot;ec2&amp;quot;, &amp;quot;s3&amp;quot;],
    &amp;quot;CI/CD&amp;quot;: [&amp;quot;continuous integration&amp;quot;, &amp;quot;github actions&amp;quot;, &amp;quot;jenkins&amp;quot;]
  }
}
```

A.10 Sample Test Job Description (sample_job_description.txt)

```
Target Role: Junior Full Stack Python Developer
Company: Apex Cloud Solutions

Requirements (Must Have):
- Bachelor&amp;#x27;s degree in Computer Applications (BCA) or equivalent.
- Strong proficiency in Python, Flask, and building RESTful APIs.
- Experience with relational databases, specifically PostgreSQL or MySQL.
- Familiarity with version control using Git and GitHub.
- Basic understanding of containerization using Docker.

Preferred (Nice to Have):
- Experience with React.js or modern frontend development.
- Familiarity with Automated Testing (PyTest) and CI/CD pipelines.
- Good problem-solving and communication skills.
```

A.11 Sample Synthetic Candidate Resume (sample_resume.txt)

```
John Doe
john.doe@example.com | +91 98765 43210 | Greater Noida, India
LinkedIn: linkedin.com/in/johndoe-bca | GitHub: github.com/johndoe

EDUCATION
Bachelor of Computer Applications (BCA) - Core
Galgotias University, Greater Noida (2024 - 2027) | Current CGPA: 8.6/10

TECHNICAL SKILLS
- Programming Languages: Python, JavaScript, C++, HTML5, CSS3
- Web Frameworks &amp;amp; APIs: Flask, RESTful APIs, Bootstrap 5
- Databases: PostgreSQL, SQLite, MySQL
- Tools &amp;amp; Version Control: Git, GitHub, VS Code, Postman
- Soft Skills: Problem Solving, Team Collaboration, Communication

ACADEMIC PROJECTS
1. E-Commerce Backend Service (Flask, PostgreSQL, Git)
- Architected RESTful endpoints for user authentication, product catalog, and order processing.
- Utilized PostgreSQL for relational storage and Git for branching and versioning.

2. Student Portfolio Web Application (HTML5, CSS3, JavaScript)
- Created responsive user interface with modern UI/UX design and JavaScript validation.

3. Task Tracker API (Python, SQLite)
- Developed lightweight microservice with complete CRUD operations.
```

APPENDIX B
HOW TO RUN THE PROJECT

APPENDIX B: HOW TO RUN THE PROJECT

B.1 Project Directory Structure

The application repository is organized in a clean, maintainable directory hierarchy:

resume-analyzer/

■

■■■ app.py # Main Flask web controller and REST API routes

■■■ resume_parser.py # Multi-format document stream extractor (PDF/DOCX)

■■■ text_preprocessor.py # Regex cleaner, sanitizer, and contact extractor

■■■ skill_extractor.py # Boundary-safe taxonomy matcher and synonym resolver

■■■ job_analyzer.py # Job description requirement parser

■■■ skill_matcher.py # Mathematical scoring and skill coverage calculator

■■■ gap_detector.py # Gap detection and learning roadmap generator

■■■ skills_taxonomy.json # Curated ontology of 750+ normalized skills & aliases

■■■ requirements.txt # Python dependency specification

■

■■■ static/

■ ■■■ css/

■ ■ ■■■ style.css # Custom responsive styles and animations

■ ■■■ js/

■ ■■■ app.js # Client-side AJAX submission and DOM updater

■

■■■ templates/

■ ■■■ index.html # Bootstrap 5 responsive web dashboard

■

■■■ temp_uploads/ # Ephemeral directory for processing uploads

■

■■■ tests/

■■■ test_suite.py # Automated unit test suite verifying all modules

B.2 Step-by-Step Execution Guide

Step 1: Environment Preparation Ensure Python 3.10 or higher is installed on the host machine. Verify installation: ```bash python --version ```

Step 2: Create and Activate Virtual Environment Isolate dependencies by creating a local virtual environment: ```bash # Windows (PowerShell / Command Prompt) python -m venv venv venv\Scripts\activate

Linux / macOS python3 -m venv venv source venv/bin/activate ```

Step 3: Install Required Dependencies Install all required libraries from the `requirements.txt` file: ```bash pip install -r requirements.txt ```

Step 4: Download spaCy Language Model Download the lightweight English language model pipeline: ```bash python -m spacy download en_core_web_sm ```

Step 5: Launch the Application Run the Flask server: ```bash python app.py ``` The terminal will display the active WSGI server banner: ```text * Serving Flask app 'app' * Debug mode: off * Running on http://127.0.0.1:5000 (Press CTRL+C to quit) ```

Step 6: Access the Web Dashboard Open any modern web browser (Google Chrome, Microsoft Edge, Mozilla Firefox) and navigate to: ```text http://127.0.0.1:5000 ```

Step 7: Analyze a Resume 1. Drag and drop a sample resume (.pdf or .docx) into the upload dropzone. 2. Select a target job preset (e.g., "Junior Full Stack Python Developer") or paste a custom job description into the text area. 3. Click the green "Analyze Resume & Detect Gaps" button. 4. The dashboard will compute results and render the compatibility gauge, matched skills badges, missing skill gaps, and the upskilling roadmap in under 0.5 seconds.

B.3 Troubleshooting Common Issues

Issue: `ModuleNotFoundError: No module named 'fitz'`

Resolution: Run `pip install PyMuPDF`. Ensure you do not install the legacy `fitz` package.

Issue: `Error: No text extracted from document`

Resolution: Confirm that the PDF contains live, selectable text rather than a flattened raster bitmap image scan.

Issue: Port 5000 already in use

Resolution: Change port in `app.py` from 5000 to 5050 (`app.run(port=5050)`).

REFERENCES

[1] D. Jurafsky and J. H. Martin, Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition, 3rd ed. draft, Prentice Hall, 2024.

[2] S. Bird, E. Klein, and E. Loper, Natural Language Processing with Python: Analyzing Text with the Natural Language Toolkit, O'Reilly Media, 2009.

[3] M. Honnibal and I. Montani, "spaCy 2: Natural language understanding with Bloom embeddings, convolutional neural networks and incremental parsing," Sentometrics Research, 2017. [Online]. Available: https://spacy.io

[4] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.

[5] Pallets Projects, "Flask Documentation (Version 3.1.x)," 2024. [Online]. Available: https://flask.palletsprojects.com

[6] Artifex Software, "PyMuPDF Documentation (Version 1.24.x)," 2024. [Online]. Available: https://pymupdf.readthedocs.io

[7] J. Singer, "pdfplumber: Plumb a PDF for detailed information about each text character, rectangle, and line," 2024. [Online]. Available: https://github.com/jsvine/pdfplumber

[8] S. Canny, "python-docx Documentation," 2024. [Online]. Available: https://python-docx.readthedocs.io

[9] European Commission, "European Skills, Competences, Qualifications and Occupations (ESCO) Taxonomy Portal," 2023. [Online]. Available: https://esco.ec.europa.eu

[10] U.S. Department of Labor, "O*NET OnLine: Occupational Information Network Database," 2024. [Online]. Available: https://www.onetonline.org

[11] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient Estimation of Word Representations in Vector Space," in Proc. International Conference on Learning Representations (ICLR) Workshop, 2013.

[12] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP), 2019, pp. 3982-3992.

[13] Stack Overflow, "2024 Developer Survey: Technology and Skill Trends," 2024. [Online]. Available: https://survey.stackoverflow.co/2024

[14] United Nations, "Sustainable Development Goals: Goal 8 - Decent Work and Economic Growth," United Nations Department of Economic and Social Affairs, 2023.

[15] P. Parasuraman and V. S. Kumar, "Automated Resume Screening and Skill Extraction Using Natural Language Processing: A Systematic Survey," IEEE Access, vol. 10, pp. 45210-45228, 2022.